Ukrainian Catholic University

Bachelor Thesis

Autonomous Ground Navigation and Dynamic Obstacle Avoidance for Husky Robot in Pedestrian Areas

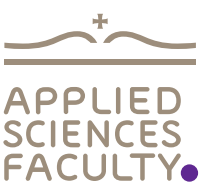
Author:
Radomyr Husiev

Supervisors:
Oleg Farenyuk
Oleksandr Kosovan

A thesis submitted in fulfillment of the requirements for the degree of Bachelor of Science

in the

Department of Computer Sciences and Information Technologies
Faculty of Applied Sciences



Lviv 2026

"*Behold, I go forward, but he is not there; and backward, but I cannot perceive him: On the left hand, where he doth work, but I cannot behold him: he hideth himself on the right hand, that I cannot see him: But he knoweth the way that I take: when he hath tried me, I shall come forth as gold.*"

Job 23:8–10 (KJV)

UKRAINIAN CATHOLIC UNIVERSITY

Faculty of Applied Sciences

Bachelor of Science

Autonomous Ground Navigation and Dynamic Obstacle Avoidance for Husky Robot in Pedestrian Areas

by Radomyr Husiev

Abstract

Autonomous outdoor navigation for Unmanned Ground Vehicles (UGVs) traditionally relies on resource-intensive perception pipelines, such as Visual SLAM and 3D LiDARs. Because these algorithms demand substantial memory and processing power, they are often impractical for lightweight platforms. This thesis therefore investigates whether a minimal sensor suite (2D LiDAR, GPS, IMU, encoders) and a CPU-optimized planner are sufficient for safe, reliable navigation in semi-structured environments.

The proposed pipeline eliminates runtime mapping by utilizing OpenStreetMap (OSM) and A* global routing. Localization relies on a dual Extended Kalman Filter (EKF), while 2D LiDAR scans are abstracted into geometric primitives for obstacle avoidance. Trajectories are optimized via a custom Python Timed Elastic Band (TEB) Ceres implementation, seeded by a dynamic visibility-graph A* search.

Evaluated through dynamic simulations of Stryiskyi Park and validated on a Clearpath Husky UGV, the system achieved an 83.3% success rate. It maintained a 10 Hz control frequency using only 293 MB of RAM and under 13% CPU utilization on a mobile processor. These results demonstrate that reliable autonomous navigation is achievable without heavy 3D perception, establishing a cost-effective paradigm for UGV deployment.

Source code: `https://github.com/goof-an-odd-husky`.

Acknowledgements

This project would not have been possible without the help of my supervisors, Oleg Farenyuk and Oleksandr Kosovan. I appreciate their time, their patience with my drafts, and their technical direction.

A special thanks goes to Hordii Yeliseyev for helping me navigate the hardware configuration for the Clearpath Husky, which was essential for the real-world validation of this system.

Thanks to Botizen, The Association of Robot Citizens, for supporting this research.

Lastly, I want to thank my family and my classmates at UCU for keeping me sane throughout the process.

Contents

Abstract **ii**

Acknowledgements **iii**

1 Introduction **1**
1.1 Background and Motivation 1
1.2 Objectives 2
1.3 Structure Of The Thesis 2

2 Related Work **3**
2.1 Path and Trajectory Planning 3
2.2 Localization 5
2.3 Semantic Mapping 6
2.4 Obstacle Abstraction 6
2.5 Conclusions 7

3 Theoretical Background **9**
3.1 Sensor Characteristics 10
3.2 Differential Drive UGV 10
3.3 Hierarchical Navigation Architecture 11
3.4 Spatial Representation 12
3.4.1 Coordinate Systems 12
3.4.2 Graph-Based Map Representation 12
3.5 Trajectory Optimization 13
3.5.1 Obstacle Cost 13
3.5.2 Kinematic Cost 14
3.5.3 Dynamics Costs 14
3.5.4 Time Cost 15
3.5.5 Solvers and Jacobians 15
3.5.6 Temporal Coherence 15
3.6 Sensor Fusion 16
3.6.1 The Kalman Filter 16
3.6.2 The Extended Kalman Filter 16
3.7 Geometric Feature Extraction 17
3.7.1 Pre-processing and Clustering 17
3.7.2 Shape Classification 18
3.7.3 Line Fitting 18
3.7.4 Circle Fitting 18

4 Proposed Solution **19**
4.1 Problem Formulation 19
4.2 Hardware and Environment Setup 19
4.3 System Architecture 20

4.4 Software Implementation . . . 20
4.4.1 Semantic Mapping and Global Routing . . . 20
4.4.2 Obstacle Abstraction . . . 21
Pre-processing and Clustering . . . 21
Shape Classification . . . 21
Primitives Extraction . . . 21
Virtual Boundaries . . . 22
4.4.3 Localization and State Estimation . . . 22
4.4.4 Local Trajectory Optimization . . . 22
Local Goal Selection and Path Initialization . . . 22
Custom Formulations of TEB Residuals . . . 23
Jacobians and Ceres Problem Setup . . . 24
Latency Compensation . . . 24
Architectural Limitations . . . 25
4.4.5 Parameter Selection . . . 25
Spatial and Obstacle Constraints . . . 25
Kinodynamic Limits . . . 26
Computational Constraints . . . 26
4.4.6 Safety Mechanisms . . . 26
4.5 Ethical and Safety Considerations . . . 27
4.6 Experimental Methodology . . . 28
4.6.1 Simulation Environment Setup . . . 28
4.6.2 Obstacle Modeling and Test Scenario . . . 28
4.6.3 Evaluation Metrics . . . 29

5 Experiments and Results **31**
5.1 Experimental Objective . . . 31
5.2 Navigation Results . . . 31
5.2.1 Travel Time and Path Efficiency . . . 32
5.3 Computational Performance . . . 32
5.4 Failure Mode Analysis . . . 33
5.4.1 Squeeze and Deadlock Failures . . . 33
5.4.2 Rear Blind-Spot Collision . . . 33
5.5 Qualitative Hardware Validation . . . 33
5.6 Conclusions . . . 34

6 Conclusions **35**

Bibliography **37**

List of Figures

2.1 Conceptual difference between reactive planning (left) and predictive planning (right). 5

3.1 Functional data flow. 9
3.2 Difference between Occupancy Grid and Semantic Map. 12
3.3 Representation of the local trajectory as a sequence of poses (x_i, y_i, θ_i) connected by time intervals Δt_i. 13
3.4 The differential-drive kinematic constraint enforces constant curvature between successive configurations, requiring the angle ϑ_i to equal ϑ_{i+1}. 14
3.5 Abstracting raw 2D LiDAR data into geometric primitives. Successive points separated by a gap larger than the threshold ($d > d_{break}$) are split into distinct clusters. Isolated points are rejected as noise using a median filter, further breaking sequential segments. 17

4.1 The Clearpath Husky A200 UGV utilized as the experimental platform. 19
4.2 Custom spatial heuristic for circular primitive extraction. 22

List of Tables

2.1 Taxonomy of relevant autonomous navigation methodologies and their trade-offs. 8

4.1 Dynamic Obstacle Course Parameters 29

5.1 Summary of Navigation Outcomes 31
5.2 Travel Time and Path Efficiency for Successful Runs 32
5.3 Computational Resource Usage and Control Frequency 32

List of Abbreviations

ACO	**A**nt **C**olony **O**ptimization
APF	**A**rtificial **P**otential **F**ields
BLE	**B**luetooth **L**ow **E**nergy
CPU	**C**entral **P**rocessing **U**nit
DWA	**D**ynamic **W**indow **A**pproach
DWB	**D**ynamic **W**indow Approach Controller
EKF	**E**xtended **K**alman **F**ilter
ENU	**E**ast-**N**orth-**U**p
GA	**G**enetic **A**lgorithms
GB / MB	**G**iga**b**yte / **M**ega**b**yte
GIL	**G**lobal **I**nterpreter **L**ock
GPS	**G**lobal **P**ositioning **S**ystem
GPU	**G**raphics **P**rocessing **U**nit
Hz	**H**ert**z**
IAV	**I**nscribed **A**ngle **V**ariance
IMU	**I**nertial **M**easurement **U**nit
KJV	**K**ing **J**ames **V**ersion
LiDAR	**Li**ght **D**etection **a**nd **R**anging
LTS	**L**ong-**T**erm **S**upport
MPC	**M**odel **P**redictive **C**ontrol
MPPI	**M**odel **P**redictive **P**ath **I**ntegral
OSM	**O**pen**S**treet**M**ap
PRM	**P**robabilistic **R**oad**m**aps
RAM	**R**andom **A**ccess **M**emory
RGB-D	**R**ed-**G**reen-**B**lue **D**epth
ROS	**R**obot **O**perating **S**ystem
RRT	**R**apidly-exploring **R**andom **T**rees
SLAM	**S**imultaneous **L**ocalization **a**nd **M**apping
TEB	**T**imed **E**lastic **B**and
UGV	**U**nmanned **G**round **V**ehicle
UI	**U**ser **I**nterface
UWB	**U**ltra-**W**ide**b**and
VFH	**V**ector **F**ield **H**istogram
VI-SLAM	**V**isual-**I**nertial **S**imultaneous **L**ocalization **a**nd **M**apping
WGS84	**W**orld **G**eodetic **S**ystem 1984

List of Symbols

a_{max}	Maximum linear acceleration	$\mathrm{m\,s^{-2}}$
d	Shortest distance to an obstacle	m
$\mathbf{F}_k$	State transition Jacobian matrix	–
$\mathbf{J}_{ij}$	Jacobian matrix of partial derivatives	–
$\mathbf{K}_k$	Kalman gain matrix	–
$\mathbf{P}_k$	State covariance matrix	–
$\mathbf{Q}_k$	Process noise covariance matrix	–
R_i	Least-squares optimization residual	–
r_{obs}	Obstacle geometric radius	m
r_{safe}	Safety boundary margin	m
$\mathbf{u}_k$	Control input vector	–
v	Linear velocity	$\mathrm{m\,s^{-1}}$
w	Residual scalar weight	–
x, y	Cartesian spatial coordinates	m
$\mathbf{x}_k$	Filter state vector	–
$\mathbf{z}_k$	Filter measurement vector	–
α	Softmin smoothing parameter	–
Δt_i	Time interval between consecutive poses	s
ε_{max}	Maximum angular acceleration	$\mathrm{rad\,s^{-2}}$
θ	Robot heading (yaw) angle	rad
ϑ_i	Angle between heading and displacement vector	rad
ω	Angular velocity	$\mathrm{rad\,s^{-1}}$

To my father, who serves in the military so that I could finish this.

Chapter 1

Introduction

1.1 Background and Motivation

Mobile robotics has become increasingly important for applications involving monotonous, remote, or hazardous tasks. Manual teleoperation of such systems is often infeasible due to scalability constraints, communication latency, and signal degradation in obstructed environments. Consequently, deploying autonomous systems capable of independent path finding and target navigation provides a more robust operational paradigm.

Outdoor navigation introduces significant complexities compared to indoor environments, as systems must account for less predictable conditions, including moving obstacles, uneven terrain, and non-cooperative dynamic agents.

Currently, off-the-shelf software frameworks for autonomous navigation exist, such as the Nav2 stack in ROS [32]. However, modern outdoor systems often rely on resource-intensive sensors (like 3D LiDARs with data rates of millions of points per second, and RGB-D cameras) and complex algorithms (like Visual SLAM [45] and MPPI [53]). These approaches may provide superior accuracy in unstructured environments but demand substantial computational resources, large amounts of RAM (often $4 - -16$ GB), and sometimes GPUs. This increases costs, drains battery life, and is not always possible on simple, low-cost platforms.

Therefore, there is a strong motivation to investigate the capabilities of a minimal, lightweight sensor suite for navigation. Mitigating the reliance on computationally intensive perception pipelines enables more accessible, cost-effective, and energy-efficient robotic platforms.

Instead of generating a map from scratch using SLAM, existing semantic data such as OpenStreetMap (OSM) can be used for global routing [19]. Instead of processing 3D point clouds or images, a simple 2D LiDAR can handle immediate, dynamic obstacles like pedestrians or trees. Finally, instead of using visual odometry, the system can rely on standard GPS fused with an IMU and wheel odometry for localization [31].

Together, these lightweight components provide all the necessary data for a robot to navigate, albeit with lower spatial fidelity. This thesis aims to demonstrate that such a minimal setup allows for safe, real-time, collision-free point-to-point navigation in a semi-structured outdoor park environment, achieving high success rates while respecting strict CPU and memory bounds.

The setup includes a differential-drive ground robot in an outdoor park. Pedestrian paths provide a semi-structured, topologically known environment. Unlike road networks, pedestrian zones lack strict traffic rules or lane changes, significantly reducing the need for heavy semantic computer vision processing. However, unlike completely unstructured off-road environments [29], these paths have clear geometric boundaries that can be modelled and tracked. The system's objective is to navigate autonomously from a start point to a target point without human intervention.

1.2 Objectives

The primary objective of this thesis is to design, implement, and evaluate a resource-efficient autonomous navigation system for a UGV in semi-structured outdoor environments. Specifically, the research addresses the following goals:

- Develop global path planning using the A* search algorithm on an OpenStreetMap environment graph.
- Design obstacle detection by processing 2D LiDAR data to extract and classify geometric primitives (lines and circles).
- Implement local navigation by combining spatial localization (via an Extended Kalman Filter) with a hybrid planner: a dynamic visibility-graph A* search for initial collision-free pathing, followed by Timed Elastic Band (TEB) optimization for kinodynamic trajectory execution.

The system was validated in a simulated environment, with performance evaluated based on success rate, safety margins, and travel time.

1.3 Structure Of The Thesis

The remainder of this thesis is structured as follows:

- **Chapter 2 (Related Work)** surveys existing approaches to path planning, localization, semantic mapping, and obstacle abstraction, highlighting the gap in lightweight solutions for outdoor navigation.
- **Chapter 3 (Theoretical Background)** details the underlying concepts, including sensor characteristics, robot kinematics, hierarchical navigation, trajectory optimization, sensor fusion, and geometric feature extraction.
- **Chapter 4 (Proposed Solution)** describes the hardware setup, system architecture, software implementation (including OSM processing, LiDAR abstraction, EKF localization, and custom TEB optimizer), safety mechanisms, and ethical considerations.
- **Chapter 5 (Experiments and Results)** evaluates the proposed architecture through dynamic simulation stress tests and qualitative hardware validation, analyzing navigation success rates, computational efficiency, and specific failure modes.
- **Chapter 6 (Conclusions)** summarizes the fulfillment of the research objectives, discusses the practical implications of the lightweight architecture, and outlines current system limitations alongside directions for future research.

Chapter 2

Related Work

Autonomous outdoor navigation is usually treated as a pipeline rather than a single algorithm. Surveys of mobile robot navigation and path planning distinguish perception, localization, mapping or spatial representation, global path planning, local motion planning, and low-level control as closely related subproblems [35, 21]. Building upon these, this chapter focuses on the components most relevant to the proposed lightweight architecture. Section 2.1 reviews path and trajectory planning approaches, Section 2.2 surveys localization techniques and the trade-offs between different sensing modalities, Section 2.3 discusses semantic mapping as an alternative to building maps from scratch, and Section 2.4 addresses the abstraction of raw sensor data into compact obstacle representations suitable for resource-constrained planners.

2.1 Path and Trajectory Planning

The problem of autonomous navigation is often decomposed into several layers: route search, path smoothing, local trajectory generation, trajectory tracking, and recovery behaviors. Since this thesis does not address high-level behavioral planning or traffic-rule reasoning, the discussion focuses on the two layers most relevant to the proposed system:

- Global path planning.
- Local trajectory planning and obstacle avoidance.

Higher-level mission planning is outside the scope of this work and is not discussed further.

Both global routing and local trajectory generation have been addressed through various algorithms tailored to different environmental constraints and vehicle kinematics. Surveys, such as those by Pandey et al. [35] and Karur et al. [21], categorize these approaches based on environmental assumptions (static vs. dynamic maps) and vehicle models (holonomic vs. non-holonomic), highlighting trade-offs between completeness, optimality, and real-time feasibility.

For global planning, graph-based approaches like Dijkstra's algorithm and A* are commonly used because they are reliable and optimal on static maps under an admissible heuristic [21]. When the environment is partially unknown or dynamically changing, incremental search algorithms such as D* and D* Lite allow efficient path repair by updating only affected segments rather than recomputing the entire route. For high-dimensional configuration spaces or complex non-holonomic constraints, sampling-based algorithms like Rapidly-exploring Random Trees (RRT) and Probabilistic Roadmaps (PRM) [26] probabilistically explore the space without requiring full discretization, though they often require post-processing for path smoothness.

Some works focus on meta-heuristic and bio-inspired optimization techniques, including Genetic Algorithms (GA), Ant Colony Optimization (ACO), and Firefly algorithms [21]. While these methods are adaptable and can escape local minima in complex search spaces, they often suffer from slow or premature convergence, making them computationally unpredictable and generally less suitable for strict real-time control on constrained hardware. Therefore, when the environment is already explicitly modeled as a sparse semantic OpenStreetMap graph, classic deterministic A* search remains the most direct, complete, and computationally lightweight choice for global routing.

However, navigating dynamic environments requires real-time local planning. Early methods, such as Artificial Potential Fields (APF) [22], modeled the environment as a physical field where the goal attracts and obstacles repel the robot. This approach often suffered from local minima and struggled with narrow gaps. Later, the Vector Field Histogram (VFH) [6] offered a simpler alternative with better performance, though it struggled with complex paths and kinematic constraints. Other approaches focused on dynamic environments by using collision cones to predict intersections between moving objects [11].

A widely used reactive approach is the Dynamic Window Approach (DWA) [17]. It evaluates a set of feasible velocities admissible within a short prediction horizon (typically a fraction of a second up to roughly one second, defined by the robot's deceleration limits) and chooses the best one based on goal proximity and obstacle avoidance. While DWA addresses vehicle kinematics and produces smooth motion, it struggles to generate paths in highly complex environments, since each decision is based only on the immediate velocity window rather than a longer-horizon trajectory. In the ROS Nav2 stack, DWA is available through the DWB controller, which retains the core ideas of DWA while exposing them as configurable critic functions [32].

The Elastic Band approach [38] models the global path as a deformable band that stretches around newly detected obstacles. However, it only provides a geometric path rather than a complete trajectory, meaning it does not calculate the specific speeds required to drive the route.

The Timed Elastic Band (TEB) algorithm [43] extends this idea using a Model Predictive Control (MPC) approach. Unlike purely reactive methods, MPC-style algorithms predict and optimize the trajectory over a future time window while enforcing the vehicle's physical limits (kinodynamic constraints). TEB optimizes this trajectory to balance execution time and safe distance from obstacles. Later extensions to the TEB framework have also addressed the issue of local minima in cluttered environments by maintaining and optimizing multiple candidate trajectories across distinctive topologies, utilizing lightweight sampling strategies to remain computationally feasible [41]. Alternatively, to avoid the computational overhead of maintaining multiple trajectories simultaneously, systems can employ a hybrid approach. As noted by Karur et al. [21], discrete geometric searches – such as running A* over a visibility graph constructed around obstacles [26] – can be used to find an initial collision-free topology. This geometric path then acts as a highly effective topological seed for continuous kinodynamic optimizers [46], allowing the system to reactively jump to a new valid topology whenever the current one becomes blocked.

More recently, sampling-based MPC algorithms, such as Model Predictive Path Integral Control (MPPI), have been introduced for aggressive driving [53]. Due to its performance, Nav2 has adopted MPPI as one of its advanced trajectory controllers alongside the DWB-based planners.

While MPC methods generate better trajectories than reactive methods like DWA (as conceptually illustrated in Figure 2.1, where DWA evaluates an immediate multitude

of velocity arcs versus MPC optimizing an entire continuous trajectory of future poses), they require more computational power. MPPI, in particular, requires parallel sampling. And while possible to run on CPU with acceptable control frequency, often necessitates a GPU [15], making it unsuitable for constrained hardware. Therefore, a CPU-optimized approach like TEB [42] provides a good balance between navigational performance and computational efficiency.

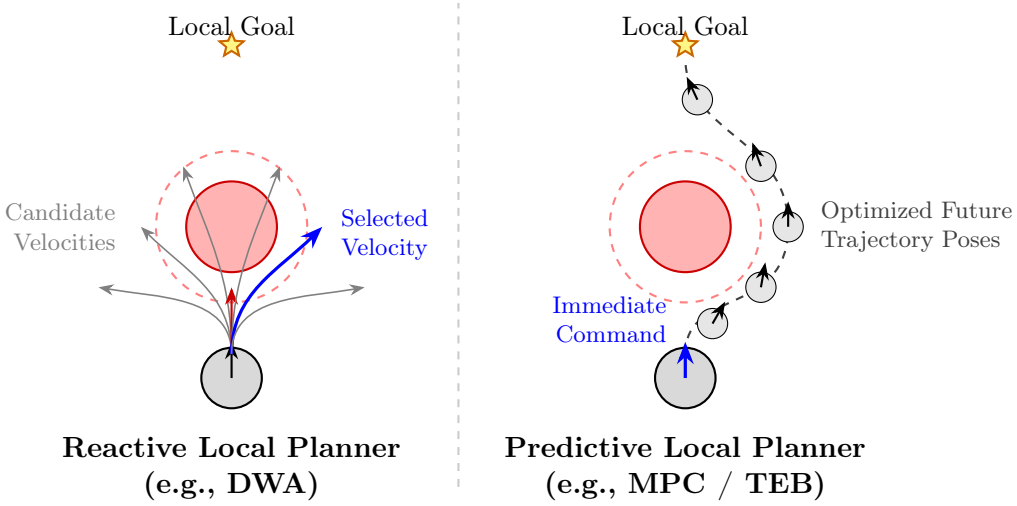


FIGURE 2.1: Conceptual difference between reactive planning (left) and predictive planning (right).

2.2 Localization

Accurate navigation requires reliable spatial localization. Currently, high-performance methods like Visual-Inertial SLAM (VI-SLAM) and 3D LiDAR-based SLAM are widely used for simultaneous localization and mapping. However, these methods are computationally heavy. For example, Schmidt et al. [45] showed that VI-SLAM algorithms can consume $4-6$ GB of RAM, and their accuracy can degrade significantly when the frame rate is reduced to save processing power. Additionally, outdoor environments like parks often cause SLAM algorithms to drift or lose tracking entirely due to moving foliage and a lack of clear structural features.

To avoid the computational overhead of full SLAM, several lightweight localization paradigms have been explored. Infrastructure-free approaches, such as 2D LiDAR scan matching against a pre-built occupancy grid (e.g., Adaptive Monte Carlo Localization) [50], are computationally efficient. However, they often degrade in semi-structured outdoor environments where repetitive paths and dynamic vegetation undermine scan registration. Radio-based methods provide an alternative family of solutions: Ultra-Wideband (UWB) and Bluetooth Low Energy (BLE) have been widely adopted for indoor positioning and fingerprinting [49]. Building on these signals, Dyhdalovych et al. [14] demonstrated a particle-filter fusion of BLE and IMU data that reduces indoor localization errors by up to 25 %.

While effective in their target domains, these alternatives have strict environment requirements. Radio-based methods need pre-installed anchors, beacons, or extensive site surveys, while LiDAR-only approaches assume a prior scanned map of the location. Both assumptions conflict with the goal of deploying a system in a public park without any environment-specific setup.

Consequently, relying on the Global Positioning System (GPS) remains the standard for outdoor navigation. Because low-cost GPS suffers from low update rates, signal obstruction, and multipath reflections under tree canopies, relying on it alone is insufficient. Therefore, fusing GPS with an Inertial Measurement Unit (IMU) and wheel odometry through an Extended Kalman Filter (EKF) serves as a highly practical alternative. Moore and Stouch [31] showed that while wheel odometry drifts over time, fusing it with IMU data improves short-term accuracy, and adding GPS further reduces long-term drift. This provides reliable, infrastructure-free localization without the heavy overhead of SLAM.

2.3 Semantic Mapping

SLAM is not only intended for localization but also for mapping. To further avoid the computational costs of SLAM, researchers have explored using existing semantic maps for global planning. Min et al. [29] note a growing trend of reducing SLAM dependency by navigating via pre-existing semantic data. Other studies highlight that explicit grid maps may not even be necessary, as internal visual odometry can sometimes provide high accuracy on its own [36].

OpenStreetMap (OSM) is a crowdsourced geographic database with tags describing road types, surface materials, and geometry. Hentschel and Wagner [19] showed that autonomous navigation can be successfully performed using OSM geodata. A distinct benefit of OSM over SLAM is its semantic context: it distinguishes footpaths from grass and provides surface details rather than just labeling space as free or occupied. Furthermore, Stahr et al. [48] use OSM attributes like path width to create boundaries for local planners, which can be used to limit the search space for trajectory optimization algorithms.

2.4 Obstacle Abstraction

Even when global routing relies on a compact semantic map rather than a dense occupancy grid, processing raw sensor data for immediate obstacle avoidance remains resource-intensive. A 3D LiDAR produces a massive number of points and introduces the problem of filtering out the ground so the road itself is not treated as an obstacle [10]. This makes 3D LiDAR computationally expensive.

Using a 2D LiDAR removes the need to reason about the vertical structure of the scene – since all measurements lie in a single horizontal plane, ground filtering and full 3D segmentation are no longer required – but still produces hundreds of points per scan. While simpler ultrasonic sensors exist, they struggle with angled surfaces and lack precise direction [5]. To optimize 2D LiDAR processing, point clouds are often abstracted into geometric primitives. Catapang and Ramos [9] and Nguyen et al. [33] grouped points within a specific distance into clusters. To extract shapes, Sezer and Gokasan [47] used circle approximations. For line extraction, Nguyen et al. [33] compared several common line-extraction algorithms and reported that "Split and Merge" provided the most efficient method for converting raw LiDAR points into lines, while still being very accurate, which has made it a common choice in subsequent work.

Dealing with dynamic agents like pedestrians is another challenge. While some approaches treat pedestrians differently from static obstacles, they often rely on heavy Reinforcement Learning and 3D tracking [13], which conflicts with computational

efficiency. However, Arras et al. [3] demonstrated that detecting dynamic agents in 2D range data is possible using only static geometric features.

To further optimize feature detection, Xavier et al. [55] introduced an approach with $O(n)$ linear complexity based on Inscribed Angle Variance (IAV). The linear complexity allows a system to categorize features into straight lines, arcs, and legs with low per-scan overhead, making it suitable for real-time use even on modest hardware.

2.5 Conclusions

As the Table 2.1 illustrates, while extensive research has produced a wide array of high-fidelity algorithms, many modern advancements heavily rely on resource-intensive perception and computation. As a result, the performance bounds of minimal, low-cost sensor suites in unpredictable outdoor environments remain underexplored.

Based on these trade-offs, the goal of this research is to build and evaluate a resource-efficient navigation system. By replacing SLAM with semantic OpenStreetMap data, abstracting 2D LiDAR data into geometric obstacles, and combining discrete visibility-graph search with CPU-optimized TEB algorithms, this thesis aims to show that the selected minimal configurations can demonstrate feasible autonomous navigation in complex outdoor environments.

TABLE 2.1: Taxonomy of relevant autonomous navigation methodologies and their trade-offs.

Problem Domain	**Dominant Paradigms**	**Key Trade-offs**	**Adopted Approach**
Localization (Sec. 2.2)	• Dense SLAM (VI/3D)	Highly accurate; but RAM/compute heavy and prone to drift in dynamic foliage.	
	• Infrastructure (UWB/BLE)	Lightweight; but requires pre-installed beacons (infeasible for public parks).	EKF Fusion (GPS + IMU + Odom.)
	• Sensor Fusion (GPS+IMU)	Infrastructure-free and low-compute; but vulnerable to GPS multi-path errors.	
Global Mapping (Sec. 2.3)	• Occupancy Grids	High geometric fidelity; but large memory footprint and requires prior exploration.	Semantic Graphs (OpenStreetMap)
	• Semantic Graphs (OSM)	Compact, uses pre-existing data, easy A* routing; but lacks real-time geometric updates.	
Obstacle Perception (Sec. 2.4)	• 3D Point Clouds	Full spatial context; requires heavy ground-filtering and high data bandwidth.	2D Geometric Primitives
	• 2D Feature Abstraction	Drastic data reduction ($O(n)$ complexity); but loses vertical awareness (blind spots).	
Local Planning (Sec. 2.1)	• Reactive (e.g., DWA)	Computationally cheap; but short-sighted and struggles in complex local minima.	Predictive (TEB) seeded by topological A*
	• Sampling MPC (e.g., MPPI)	Aggressive and highly capable; but requires parallelization (often GPU).	
	• Optimization MPC (e.g., TEB)	Evaluates full trajectory, respects kinodynamics, CPU-friendly; but can deadlock in one topology.	

Chapter 3

Theoretical Background

This chapter presents the theoretical foundations underlying the proposed lightweight navigation system. The discussion follows the structure of the system itself, moving from low-level sensing to high-level decision-making. Section 3.1 reviews the characteristics and limitations of the minimal sensor suite. Section 3.2 describes the kinodynamic properties of the four-wheeled skid-steer UGV. Section 3.3 introduces the hierarchical decomposition into global and local planning. Section 3.4 discusses how the world is represented as a semantic graph and how A* operates on it. Section 3.5 formulates local trajectory generation as a non-linear least-squares problem. Section 3.6 describes how heterogeneous, noisy measurements are merged through Kalman filtering. Finally, Section 3.7 explains how raw 2D LiDAR scans are abstracted into geometric primitives.

The functional data flow of this entire pipeline is illustrated in Figure 3.1. As shown, raw inputs (OSM, 2D LiDAR, and state sensors) are independently compressed into high-level abstractions—such as sparse graphs, geometric clusters, and dual-EKF pose estimates—before converging at the local trajectory optimizer.

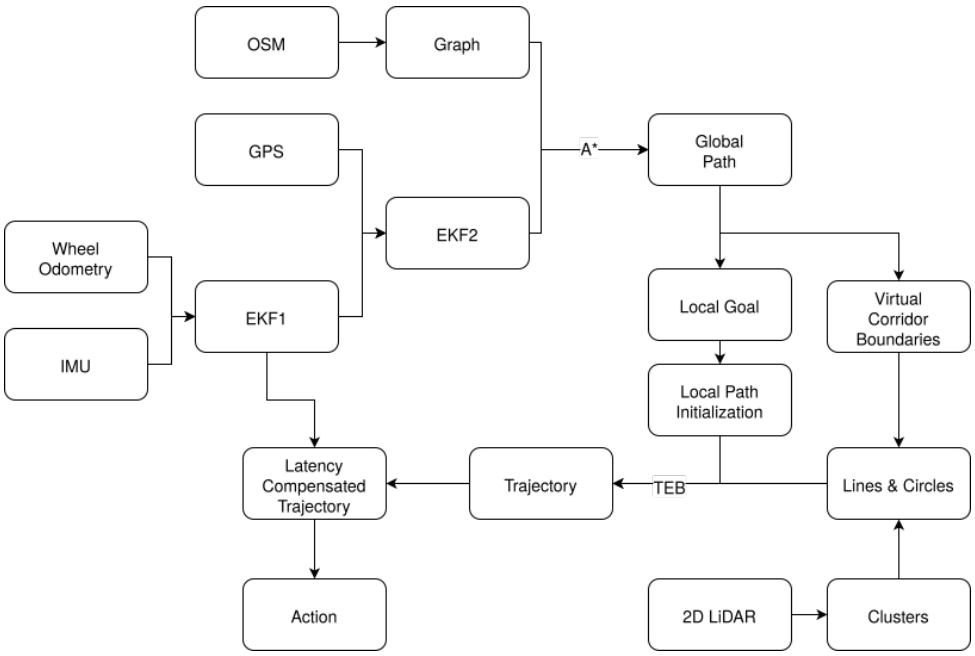


FIGURE 3.1: Functional data flow.

Together, these components form a pipeline in which each layer reduces the volume of raw data passed to the next: raw LiDAR scans are abstracted into a compact set of geometric primitives, multi-sensor streams are fused into a single pose estimate, and a global graph containing thousands of nodes is compressed into a short sliding window of waypoints for the optimizer.

3.1 Sensor Characteristics

Autonomous navigation systems rely on multiple sensors to perceive the environment and estimate the vehicle's state. Because a core objective of this research is resource efficiency, it is important to understand the capabilities and limitations of the minimal sensor suite. The numerical values below are estimates for low-cost or research-grade mobile robot sensors:

- Wheel Encoders: Estimate the robot's relative displacement by measuring wheel rotation. The built-in encoders provide a resolution of 78 000 ticks/m [12]. While useful for short-term tracking, odometry accumulates drift over time due to mechanical slip, which is especially prominent in skid-steer UGV configurations [28].
- Inertial Measurement Unit (IMU): Contains accelerometers and gyroscopes to measure linear acceleration, and angular velocity, typically up to 250 Hz [37]. Sometimes a magnetometer data is included. It provides frequent motion updates but suffers from sensor bias and integration drift. As characterized by Woodman [54], gyroscope bias produces orientation error that grows linearly with time, while uncorrected accelerometer bias causes velocity error to grow approximately linearly and position error to grow quadratically. Orientation errors further induce horizontal acceleration errors through gravity misprojection, leading to rapidly growing position drift [54]. Even consumer-grade IMUs can therefore accumulate position errors of tens of meters within a minute of standalone integration.
- Global Positioning System (GPS): Provides absolute geographic positioning (latitude, longitude) by trilaterating satellite signals. Consumer-grade receivers typically operate at $1 - 10$ Hz with a horizontal accuracy of several meters under open sky [51]. Accuracy degrades due to signal obstruction, multipath reflections from surfaces such as trees or buildings, and electromagnetic interference [45].
- 2D LiDAR (Light Detection and Ranging): Uses laser pulses to measure distances in a single horizontal plane, outputting a point cloud in polar coordinates. Typical outdoor-rated scanners operate at tens of hertz, yielding range measurements in thousands per scan with a maximum range of tens of meters [20]. While computationally much lighter than 3D LiDAR (which produces hundreds of thousands of points per second), it lacks vertical resolution and cannot distinguish between traversable terrain features and vertical obstacles outside its scanning plane.

These sensors have complementary properties. Encoders and IMUs provide frequent relative motion estimates but drift over time. GPS provides absolute position but is noisy and slow. The 2D LiDAR provides local obstacle measurements but only in one plane. The navigation architecture must therefore combine them through sensor fusion and local obstacle abstraction.

3.2 Differential Drive UGV

The experimental platform utilizes a four-wheeled skid-steer differential drive configuration. Steering is achieved through a velocity differential between the left and right wheel pairs, introducing specific kinematic constraints.

To generate realistic and executable trajectories, autonomous planners must respect the physical limitations of the robot, which are broadly categorized into kinematic and dynamic constraints. Together, these define the robot's kinodynamic constraints.

- Kinematic constraints. A differential drive system acts under non-holonomic kinematic constraints: it can rotate in place and move forward or backward, but it cannot move laterally without sliding. Therefore, the robot is controlled via its linear (v) and angular (ω) velocities [30]. If a planner generates a path that requires direct lateral movement, the hardware physically cannot execute it.

- Dynamic constraints. The UGV has physical mass, limited motor torque, and a fixed gear reduction in its drivetrain, so it cannot change velocities instantly. Dynamic constraints therefore define the robot's maximum achievable velocities (v_{max}, ω_{max}) and maximum accelerations (a_{max}, ε_{max}), which are themselves bounded by the available wheel torque and the gearbox ratio between the motors and the wheels.

The four-wheeled skid-steer configuration further complicates motion estimation. Unlike two-wheeled differential systems, all four wheels are fixed in parallel: turning requires the motors to overcome static friction and physically drag the tires laterally across the terrain. The magnitude of this slip depends on surface friction, payload distribution, and turn radius, but it generally introduces additional drift into wheel-based odometry compared to ideal two-wheeled differential models [28]. Accurate sensor fusion therefore becomes essential to correct for this drift.

3.3 Hierarchical Navigation Architecture

A widely adopted architecture for autonomous mobile robot navigation decomposes the problem into a two-level hierarchy of global and local planning [35, 21]. This decomposition is motivated by several factors. First, the navigation task is naturally hierarchical: the high-level question of which roads to follow is qualitatively different from the low-level question of how to steer around a pedestrian who just stepped into the path. Second, the global environment is largely static and known in advance, while the local surroundings are dynamic and only partially observable [21]. Third, computing a single kinodynamically feasible, collision-free trajectory all the way from the current pose to a distant goal would scale poorly with map size and would have to be recomputed from scratch every time a new obstacle appears.

For these reasons, the classical two-level decomposition is adopted in this work. The global planner operates on a static map to find a route to the destination. Although such a route is geometric in nature – it consists of waypoints with metric coordinates and edge lengths – it is computed without considering the vehicle's instantaneous kinematics or transient dynamic obstacles, and is in that sense closer to a topological plan than to an executable trajectory. The local planning stage is often further subdivided into geometric path finding and kinodynamic optimization. First, an intermediate geometric path is dynamically routed around newly perceived obstacles using a discrete search (e.g., A* on a visibility graph). Second, a trajectory optimizer (e.g., TEB) translates this collision-free geometric path into executable motion commands that respect the vehicle's kinodynamic limits in real time.

3.4 Spatial Representation

3.4.1 Coordinate Systems

GPS sensors output data in geodetic coordinates (latitude, longitude, altitude). Trajectory planners operate in Cartesian coordinates. Geodetic coordinates must be projected onto a local tangent plane in order for the local planner to calculate distances in meters.

3.4.2 Graph-Based Map Representation

For a global planner to generate paths, the environment is abstracted into a directed semantic graph [19]. Unlike grid-based maps that represent the world as a dense matrix of occupied or free cells, a semantic graph explicitly models the topology and attributes of the environment. Vertices represent discrete spatial coordinates (such as path intersections), while edges represent the physical paths connecting them. The semantic nature of the graph means each edge contains metadata (such as surface type or access restrictions). This allows the system to mathematically prune non-navigable edges – such as unpaved terrain – before routing begins. For the remaining valid edges, complex paths made of multiple physical segments are simplified into single edges connecting these intersections. The edge weight for such paths is defined as the sum of its underlying segment lengths. This preserves metric distance accuracy while drastically reducing the number of vertices the planning algorithm must evaluate. A visual comparison of this abstraction is provided in Figure 3.2, contrasting a dense, cell-based occupancy grid with a sparse, node-and-edge semantic representation.

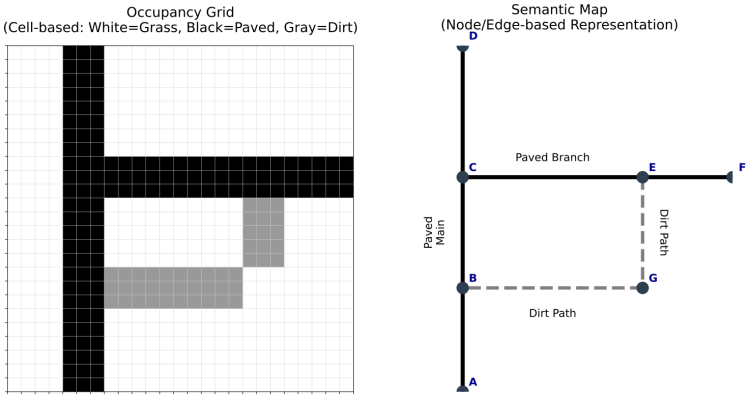


FIGURE 3.2: Difference between Occupancy Grid and Semantic Map.

The A* algorithm [18] navigates this graph. It computes a least-cost path by minimizing the function $f(n) = g(n) + h(n)$, where $g(n)$ is the exact accumulated cost from the start vertex to vertex n and $h(n)$ is the estimated heuristic cost from n to the goal. For A* to be guaranteed to return an optimal path with respect to the chosen edge cost, the heuristic must be admissible, meaning it never overestimates the actual cost to the goal [40]. Because a straight line is the shortest possible distance between two points in Euclidean space, the Euclidean heuristic is admissible whenever edge costs correspond to physical distance.

In this work, edge costs equal segment lengths, so A* returns the shortest geometric route. However, the same framework supports richer cost functions: edges may be weighted by expected travel time, surface quality (favoring paved over unpaved terrain) or the side of the pavement (preferring the right side) [48]. The choice of edge cost

simply has to remain consistent with an admissible heuristic for optimality guarantees to hold.

By prioritizing nodes with lower $f(n)$ values, the algorithm preferentially expands vertices oriented toward the target, rather than propagating uniformly in all directions as Dijkstra's algorithm would. This significantly reduces the number of vertices processed while preserving path optimality.

3.5 Trajectory Optimization

The Timed Elastic Band (TEB) approach [43] formulates local navigation as a multi-objective optimization problem with a receding-horizon strategy. At each control step, the algorithm predicts a short-term trajectory over a finite time window, applies the first generated control command, and continuously re-optimizes the trajectory as new sensor data arrives. The trajectory itself is defined as a sequence of intermediate poses between the start and goal. Each pose consists of x, y coordinates (in meters) and a heading angle θ (in radians). Successive poses are linked by a time interval Δt_i (in seconds), representing the duration the robot needs to travel from pose i to pose $i+1$. The trajectory is initially generated as a sequence of points, with Δt_i and θ linearly interpolated. This sequence of discrete poses and time intervals is visualized in Figure 3.3.

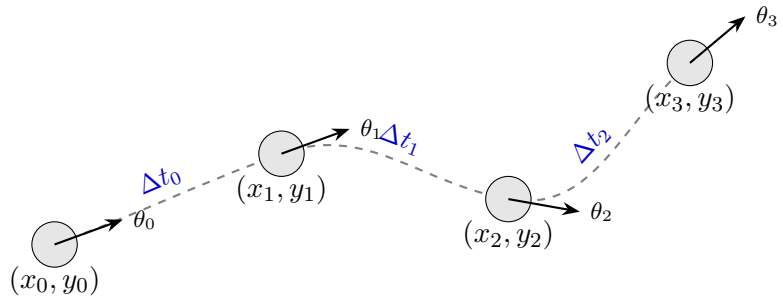


FIGURE 3.3: Representation of the local trajectory as a sequence of poses (x_i, y_i, θ_i) connected by time intervals Δt_i.

The trajectory is then deformed based on constraints and obstacles. The problem is framed as a non-linear least-squares optimization. The robot's poses and time intervals act as the parameters being optimized. The physical and spatial constraints act as cost functions, also known as residuals and denoted R_i, where each R_i is a scalar function of the trajectory parameters that equals zero when the corresponding constraint is satisfied and grows in magnitude as it is violated. The optimization engine minimizes a weighted sum of the squared residuals. In a least-squares solver, the total cost is defined as $\frac{1}{2}\sum_i R_i^2$. To apply a positive scalar weight w that scales the relative importance of one residual against the others, the raw error is multiplied by $\sqrt{w}$; squaring then recovers exactly w times the underlying squared error, ensuring the solver sees the intended weighting.

The standard formulations of these residuals [43] ensure safety, kinematic feasibility, and efficiency, as detailed below.

3.5.1 Obstacle Cost

To prevent collisions, the solver penalizes poses that fall within a safety boundary around geometric primitives. For a circular obstacle with center O and radius r_{obs},

the cost is computed from the shortest distance between the obstacle center and the line segment AB connecting two consecutive trajectory poses. Let P be the closest point on AB to O, parameterized as $P = A + t(B - A)$, where the projection scalar t is clamped to $[0, 1]$ to ensure the closest point lies strictly on the segment. With $d = \|O - P\|$ denoting that minimum distance, r_{safe} a configurable safety margin around the robot, and w the residual weight:

$$R_{obs} = \begin{cases} \sqrt{w} \cdot (r_{obs} + r_{safe} - d) & \text{if } (r_{obs} + r_{safe}) > d, \\ 0 & \text{otherwise.} \end{cases} \tag{3.1}$$

The residual is zero whenever the segment is sufficiently far from the obstacle, and grows linearly as the trajectory encroaches on the safety zone.

3.5.2 Kinematic Cost

To enforce the differential-drive constraint, the robot's motion between two configurations i and $i + 1$ is modeled as an arc segment of constant curvature [43]. This requires the angle ϑ_i between the robot's heading at pose i and the displacement vector $\mathbf{d}_{i,i+1} = (\Delta x, \Delta y)^\top$ to be equal to the corresponding angle ϑ_{i+1} at pose $i + 1$, as shown in Figure 3.4. Rather than optimizing these angles directly, the constraint is formulated as a cross product:

$$\begin{pmatrix} \cos\theta_i \\ \sin\theta_i \\ 0 \end{pmatrix} \times \mathbf{d}_{i,i+1} = \mathbf{d}_{i,i+1} \times \begin{pmatrix} \cos\theta_{i+1} \\ \sin\theta_{i+1} \\ 0 \end{pmatrix}. \tag{3.2}$$

Because the displacement vector lies in the xy-plane, the cross products reduce to a single scalar equation, yielding the resulting least-squares residual R_{kin}:

$$R_{kin} = \sqrt{w} \cdot \Big[(\cos\theta_i + \cos\theta_{i+1})\Delta y - (\sin\theta_i + \sin\theta_{i+1})\Delta x \Big]. \tag{3.3}$$

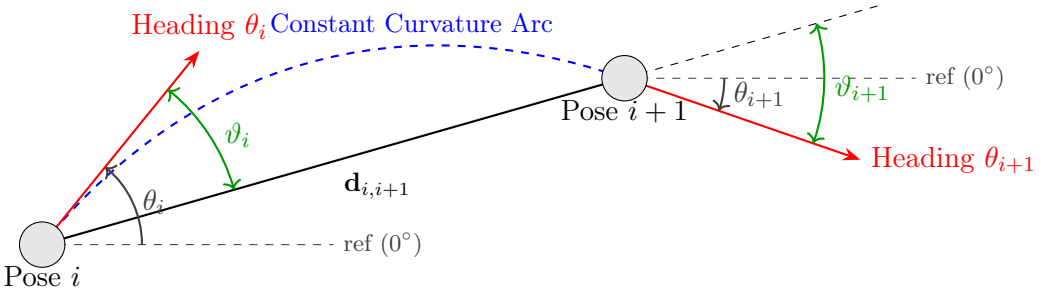


FIGURE 3.4: The differential-drive kinematic constraint enforces constant curvature between successive configurations, requiring the angle ϑ_i to equal ϑ_{i+1}.

3.5.3 Dynamics Costs

These costs penalize trajectories that exceed the UGV's physical limits. Instantaneous velocities and accelerations are not stored as parameters of the trajectory; instead, they are reconstructed from neighboring poses using finite-difference approximations. For example, the linear velocity along the segment between poses i and $i + 1$ is

approximated as

$$v_i \approx \frac{\|\mathbf{d}_{i,i+1}\|}{\Delta t_i}, \tag{3.4}$$

where $\|\mathbf{d}_{i,i+1}\|$ is the Euclidean distance between the two poses and Δt_i is the associated time interval. This estimate is then compared against the maximum allowed linear velocity v_{max}, providing the residual:

$$R_v = \begin{cases} \sqrt{w} \cdot (|v_i| - v_{max}) & \text{if } |v_i| > v_{max}, \\ 0 & \text{otherwise.} \end{cases} \tag{3.5}$$

Analogous formulations apply to angular velocity, linear acceleration, and angular acceleration, with angle differences normalized to $[-\pi, \pi]$. Two consecutive poses are required to compute the finite-difference velocity, while three poses are needed for accelerations, since they require two consecutive velocities to estimate the change in velocity over time.

3.5.4 Time Cost

To prevent the solver from minimizing other penalties by inflating time intervals (and thereby driving velocities and accelerations toward zero), a penalty is applied directly to each Δt_i. Minimizing the sum of these intervals minimizes total travel time, biasing the optimizer toward forward progress:

$$R_t = \sqrt{w} \cdot \Delta t_i. \tag{3.6}$$

Because Δt_i is constrained to be non-negative, this residual grows linearly with the duration of each trajectory segment, while the obstacle and kinematic costs simultaneously prevent the trajectory from becoming too aggressive.

3.5.5 Solvers and Jacobians

Traditionally, the TEB algorithm models the trajectory optimization as a hyper-graph, where robot poses represent the vertices and the kinematic or spatial constraints represent the edges. Represented this way, it is solved using the `g2o` graph optimization framework [25]. However, this hyper-graph formulation is mathematically equivalent to a standard non-linear least-squares optimization problem [1]. In a least-squares framework, the poses and time intervals act as parameter blocks, and the constraints act as residual blocks.

To solve this objective, optimization engines require Jacobians – matrices of partial derivatives $\mathbf{J}_{ij} = \partial R_i / \partial p_j$ indicating how changes in the trajectory parameters affect each residual. These can be computed analytically or numerically [34]. Because trajectory constraints typically couple only adjacent poses, the resulting Jacobian is highly sparse. Solvers exploit this structure with sparse factorization methods (such as sparse Cholesky decomposition) to efficiently solve the underlying linear systems, iteratively updating the parameters until they converge on a smooth, physically feasible trajectory [42, 1].

3.5.6 Temporal Coherence

If an optimizer starts from a blank slate (a cold start) at every control step, it may fail to converge within strict time limits and produce inconsistent trajectories. MPC-style approaches, including TEB, solve this by utilizing a Warm Start [42]. Because the

environment changes only slightly between control frames, the optimized trajectory from the previous time step can be preserved and used as the initial guess for the next iteration. This temporal coherence reduces the number of iterations required for the solver to find a local minimum. However, if a dynamic obstacle moves directly across the optimized trajectory, the warm start becomes invalid. In such cases, the system must perform a "cold start," re-initializing the optimizer with a newly generated geometric path that routes around the obstruction.

3.6 Sensor Fusion

Raw sensor measurements are inherently corrupted by noise and specific inaccuracies. As described in Section 3.1, wheel odometry accumulates drift from mechanical slippage, IMU measurements integrate into unbounded error [54, 31], and GPS receivers provide absolute global coordinates but suffer from high-frequency noise, signal obstruction, and multipath effects [45]. No single sensor in the minimal suite is sufficient on its own; combining them through probabilistic state estimation allows their complementary strengths to compensate for individual weaknesses.

3.6.1 The Kalman Filter

The Kalman Filter is a recursive Bayesian estimator that maintains a Gaussian belief over the system state and quantifies its uncertainty [31, 50]. It maintains two quantities: a state vector $\mathbf{x}$ encoding variables of interest (such as position, orientation, and velocity), and a covariance matrix $\mathbf{P}$ capturing the current uncertainty about that state and the correlations between its components. The filter operates in two alternating steps:

- **Prediction.** Using a motion model $f(\cdot)$ and control input $\mathbf{u}_k$, the filter projects the state forward: $\mathbf{x}_{k|k-1} = f(\mathbf{x}_{k-1}, \mathbf{u}_k)$. The covariance is propagated as $\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^\top + \mathbf{Q}_k$, where $\mathbf{F}_k$ is the state transition Jacobian and $\mathbf{Q}_k$ models process noise. The covariance grows during prediction, reflecting that integrating noisy inputs increases uncertainty. In this system, high-frequency IMU and wheel odometry data drive the prediction step.
- **Update.** When a new measurement $\mathbf{z}_k$ arrives (typically from GPS, at a much lower rate), the filter computes the Kalman gain $\mathbf{K}_k$, which optimally weights the predicted state against the new observation based on their respective uncertainties: a noisier measurement is trusted less, while a more uncertain prediction is corrected more aggressively. The state and covariance are then corrected as $\mathbf{x}_k = \mathbf{x}_{k|k-1} + \mathbf{K}_k(\mathbf{z}_k - h(\mathbf{x}_{k|k-1}))$, reducing overall uncertainty.

3.6.2 The Extended Kalman Filter

The standard Kalman Filter assumes that both the motion and measurement models are linear. However, mobile robot motion is inherently non-linear, since headings combine with linear velocities through trigonometric functions to produce changes in x and y. The Extended Kalman Filter (EKF) addresses this by linearizing the motion and measurement models around the current state estimate via a first-order Taylor series expansion, computing the Jacobians of these models, and applying the standard Kalman update equations to the linearized system [31].

A practical advantage of this fusion scheme is graceful degradation: if a sensor such as GPS becomes temporarily unavailable, the filter simply skips its update step

and continues to propagate the state through prediction. The robot can therefore keep navigating using its internal motion model alone – a fallback known as dead reckoning. However, because the prediction step has no source of absolute correction in this mode, the covariance grows monotonically, and accumulated drift will eventually exceed safe operational thresholds if the absolute signal is not restored [8].

3.7 Geometric Feature Extraction

2D LiDAR outputs data in polar coordinates (distance and angle). To be used by the optimization engine, this point cloud must be segmented and abstracted into geometric primitives (lines and circles) in Cartesian space.

3.7.1 Pre-processing and Clustering

Raw LiDAR scans frequently contain isolated noise spikes and artifacts. Therefore, the points are first passed through a median filter to reject extreme outliers. The filtered point clouds are then grouped into discrete clusters using distance-based segmentation [33]. Points are assigned to a single sequential object if the Euclidean distance between consecutive measurements P_i and P_{i+1} is below a defined break distance. The vehicle diameter, with some padding, can serve as an effective threshold, as narrower gaps would not be passable by the robot. The mechanics of this distance-based segmentation, along with outlier filtering, are visualized in Figure 3.5.

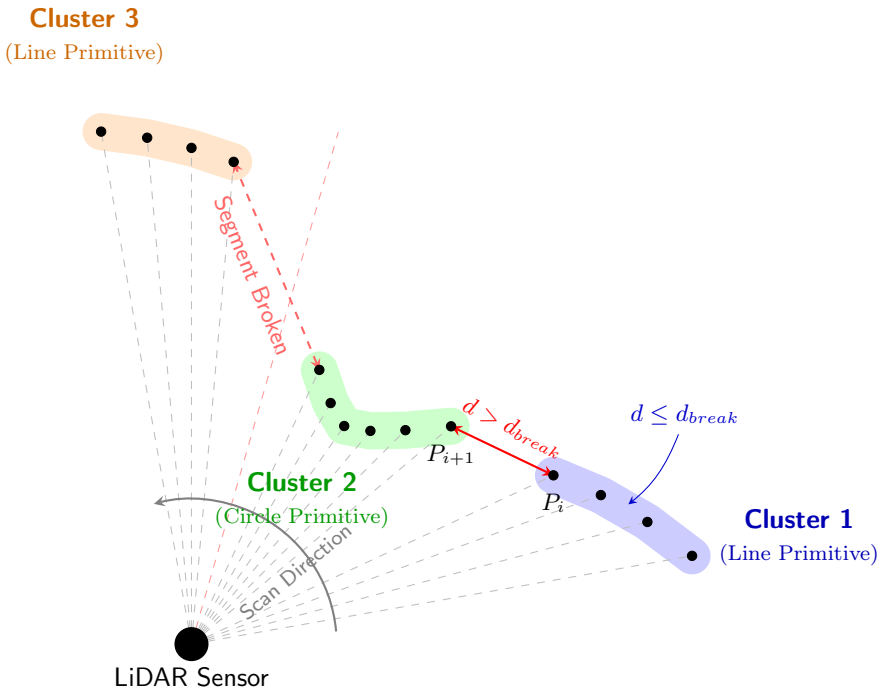


FIGURE 3.5: Abstracting raw 2D LiDAR data into geometric primitives. Successive points separated by a gap larger than the threshold ($d > d_{break}$) are split into distinct clusters. Isolated points are rejected as noise using a median filter, further breaking sequential segments.

3.7.2 Shape Classification

Clusters must be categorized as linear or circular to apply the correct geometric extraction logic. For clusters with three or more points, a geometric pre-filter [55] is used to evaluate the curvature. A chord vector is drawn between the cluster's endpoints, P_1 and P_3, and the cross product is used to calculate the perpendicular arc depth to the median point, P_2. If this depth is bounded between 10% and 70% of the chord length, the cluster exhibits sufficient convexity to be classified as a circle. Clusters falling outside this range, or clusters exhibiting an excessive maximum internal point-to-point distance, are classified as lines.

3.7.3 Line Fitting

Line extraction uses the Split-and-Merge algorithm [33]. First, a chord is drawn between the cluster's endpoints. The algorithm calculates the perpendicular distance from every internal point to the chord. If the maximum distance exceeds a predefined threshold, the cluster is split at that peak point into two sub-scans, and the process recurses. The result is a set of distinct linear segments that approximate complex shapes like walls or curbs.

3.7.4 Circle Fitting

For clusters identified as circular obstacles (such as trees or pedestrians), geometric parameters (center and radius) must be extracted. Algebraic methods, such as the circumcenter approach [55], attempt to find an analytical solution by selecting three points and computing the intersection of their perpendicular bisectors.

While mathematically exact for perfect curves, algebraic fitting frequently fails when applied to 2D LiDAR data. Because LiDAR only senses the front face of an obstacle, the returned points form a shallow arc. For distant or small obstacles, the limited resolution of the scanner causes these arcs to approach a straight line. Furthermore, because the laser beam has a physical width, reflections from the highly angled flanks of cylindrical objects register prematurely. This smearing effect flattens the perceived curve, causing traditional algebraic methods to systematically overestimate the cylinder's diameter by up to 19% [16].

Chapter 4

Proposed Solution

4.1 Problem Formulation

The objective of this research is to enable a UGV to reach a target position (defined via GPS coordinates or a relative Cartesian offset) from an initial starting point without direct human control. Building upon the theoretical models established in Chapter 3, the proposed solution is an OSM-based hierarchical autonomous navigation system designed for a semi-structured outdoor pedestrian environment.

The core constraint of this implementation is achieving reliable obstacle avoidance and spatial accuracy utilizing only a minimal sensor suite (2D LiDAR, GPS, IMU, wheel odometry) and an onboard CPU. By integrating semantic-graph-based global routing with receding-horizon local trajectory optimization, the system aims to minimize computational demands while maintaining reliable obstacle avoidance and spatial accuracy.

4.2 Hardware and Environment Setup

The experimental subject of this research, shown in Figure 4.1, is the Clearpath Husky A200 [12], a medium-sized UGV. The Husky is a rugged, four-wheeled skid-steer platform designed for research applications. It has a physical footprint of $99 \times 67 \times 39$ cm, weighing approximately 50 kg.

Figure 4.1: The Clearpath Husky A200 UGV utilized as the experimental platform.

The UGV is equipped with a minimal sensor suite and compute setup, which includes:

- Built-in Wheel Encoders, with a resolution of 78 000 ticks/m.
- IMU: PhidgetSpatial 3/3/3 Basic (Phidgets 1042).
- 2D LiDAR: Hokuyo UTM-30LX-EW, providing a 270-degree horizontal field of view.
- GPS Receiver: u-blox NEO-6M-0-001.
- Onboard Computer: MiTAC PH13FEI equipped with an Intel Core i7-9700TE processor and 16 GB of RAM, providing all the necessary CPU computation without a dedicated GPU.
- Software Stack: Ubuntu 24.04.1 LTS running ROS 2 Jazzy [27].

The proposed system was designed for Stryiskyi Park, Lviv, Ukraine. The environment is semi-structured, featuring mapped, paved pedestrian paths but lacking traffic rules or lane markings. While the park's topography includes both flat and hilly regions, this work is built for predominantly flat sections, as navigation on hilly terrains using only 2D sensors poses additional challenges in ground removal and obstacle filtering [23]. This environment poses several challenges:

- The park features dense tree coverage, which might degrade GPS signals, testing the resilience of the EKF sensor fusion.
- There are dynamic, non-cooperative entities – primarily pedestrians, bicycles, and pets – whose unpredictable trajectories test the responsiveness and real-time collision avoidance capabilities of the system.

4.3 System Architecture

The system consists of two ROS 2 nodes:

- Controller Node: Executed directly on the UGV. It processes raw sensor data, extracts geometric primitives for obstacle abstraction, and computes both the global path and local trajectory. Running this node onboard minimizes network latency and ensures real-time responsiveness.
- Visualization Node: Operates off-board on a separate machine within the same local network. It subscribes to the Controller Node to render obstacles, paths, and trajectories, displaying them in a UI. It also allows entering the desired goal location or canceling navigation.

4.4 Software Implementation

4.4.1 Semantic Mapping and Global Routing

OSM is used for a semantic map. To process this data, the Python `osmnx` library [4] is utilized. It allows for the extraction of a multigraph for a specific OSM polygon relation (such as the park boundary), where traversable paths act as edges and intersections act as vertices. To filter out non-navigable terrain, the following edge tags are evaluated:

- `surface=`. Surfaces other than `paved, asphalt, concrete, paving_stones, sett, cobblestone` are excluded.

- `highway=`. For example, `motorway` and `cycleway` are excluded.
- `access=` and `foot=`. If any of these is `no` or `private`, the road is discarded.

After initial filtering, the global path can be generated. The A* algorithm is used to get a sequence of vertices, as described in Section 3.4.2. These need to be followed in order.

The local planner does not operate on a graph; instead, it works in a Cartesian coordinate system. Therefore, the sequence of vertices has to be converted into a sequence of 2D points connected by straight lines. Because physical footpaths are rarely perfectly straight, single edges in the graph are often represented in OSM as a sequence of smaller linear segments (stored within the `geometry` attribute). By splitting each edge into these underlying segments, the global planner outputs a sequence of 2D Cartesian waypoints suitable for the local planner.

4.4.2 Obstacle Abstraction

The local trajectory planner requires distinct geometric constraints rather than a dense point cloud. To achieve this, a custom perception pipeline is implemented to process 2D LiDAR scans into Cartesian geometric primitives based on the theoretical models described in Chapter 3.

Pre-processing and Clustering

Raw LiDAR scans are converted from polar to Cartesian coordinates. Points below a minimum range (rays reflected from the vehicle itself) are discarded. The scan is passed through a 1D median filter of a specified window size to reject isolated noise. The points are then grouped sequentially, broken based on the break distance, as described in Section 3.7.1.

Shape Classification

Clusters with fewer than three points are automatically classified as circles. For larger clusters, if the maximum internal Euclidean distance exceeds the geometry split threshold, it is considered a line. For the remaining clusters, the Xavier heuristic [55] is applied as discussed in Section 3.7.2. The arc depth is calculated, and if it falls between 10% and 70% of the chord length, the cluster is classified as a circle; otherwise, it is treated as a line.

Primitives Extraction

To ensure stability against sensor noise and bias introduced with algebraic curvature methods of circle extraction, a spatial heuristic was developed, illustrated in Figure 4.2. The maximum Euclidean distance between any two points in the cluster (the chord width) establishes an initial diameter estimate. To counteract sensor noise and the systematic cylinder overestimation identified by Forsman et al. [16], the radius (half diameter) is clamped between predefined physical minimum and maximum bounds. Instead of calculating a mathematical center from the arc, the visible middle point of the cluster is projected outward from the sensor origin by the estimated radius to approximate the true geometric centroid. Finally, the radius is expanded, so that all points in the cluster are enclosed. This method mitigates overestimations while implicitly providing a safety margin. Clusters classified as lines are processed using the recursive Split-and-Merge algorithm to yield distinct linear segments, as detailed in Section 3.7.3.

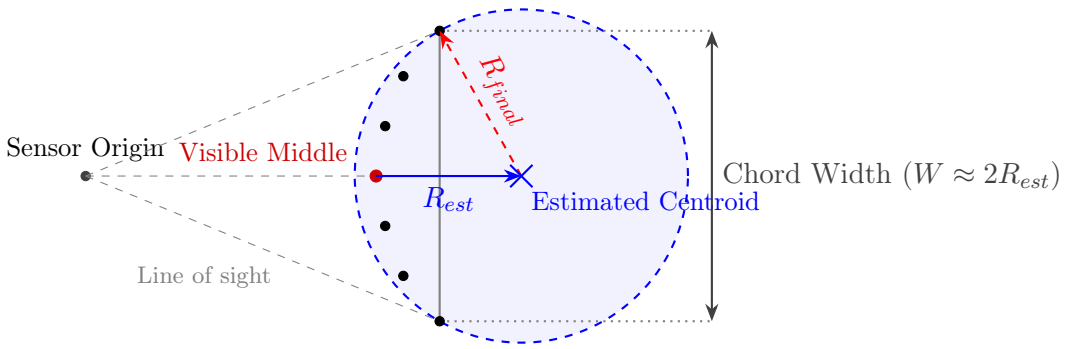


FIGURE 4.2: Custom spatial heuristic for circular primitive extraction.

Virtual Boundaries

To restrict the UGV to the pedestrian path, virtual line obstacles are generated. While path width can be extracted from OSM tags, in practice, these tags are often unmapped. Instead, the implementation uses a default value of 4 meters. Parallel line barriers are placed on both sides of the target trajectory, creating a safe routing corridor for the local planner.

4.4.3 Localization and State Estimation

An Extended Kalman Filter, described in Section 3.6, is implemented via the `robot_localization` ROS 2 package. The EKF is configured to fuse data from the internal IMU and wheel encoders for continuous state estimation. It is used for the trajectory updates, so as not to introduce sudden shifts for the TEB local planner.

Another EKF is used to take into account low-frequency GPS data. It is projected onto a local Cartesian tangent plane and fused to reduce the drift. This second filter's output is used to update the direction toward the target, ensuring the vehicle reaches the exact destination.

As previously illustrated in the functional block diagram in Figure 3.1, the first EKF (EKF1) handles the continuous wheel odometry and IMU fusion, while the second one (EKF2) incorporates the absolute GPS data atop the EKF1 baseline.

In fallback scenarios where GPS is disabled or unavailable, the direction to the global goal is only updated using the first filter, which uses IMU and wheel odometry.

4.4.4 Local Trajectory Optimization

A custom Python implementation of the Timed Elastic Band algorithm was developed, as detailed in Section 3.5, utilizing the `pyceres` library [44] for non-linear optimization. `pyceres` provides minimal Python bindings that expose the core functionality of the highly optimized C++ Ceres Solver [1], allowing the high-level logic to be orchestrated in Python while maintaining near-native execution speeds.

Local Goal Selection and Path Initialization

Because optimizing a trajectory all the way to the final destination is infeasible to compute in real time, the TEB planner operates on a sliding window. At each control loop, the UGV's current position is projected onto the global path (the closest waypoint to the current position is selected). The algorithm traverses the path forward until an accumulated length is equal to the predefined lookahead distance. This coordinate

becomes the temporary local goal for the optimizer. If the local goal falls inside or too close to an obstacle, it is slid along the global path until a free space is found.

To ensure the solver remains stable, the trajectory resolution is dynamically resized between iterations. The distances between consecutive poses are calculated. If a distance is lower than a lower bound, the pose is pruned, and the Δt interval of the previous pose is increased by the Δt of the current one. If the distance is larger than the upper bound, a new pose gets inserted in the middle of the other two, and the Δt is halved and spread across the inserted pose and the first of the two old ones.

Once the local goal is established, an initial collision-free path must be generated. While TEB is highly efficient at deforming an existing trajectory, it requires a safe initial guess. Furthermore, standard TEB cannot easily "jump" to the opposite side of an obstacle once a trajectory is established. To overcome this, a custom dynamic A* search acts as a topological initializer.

The algorithm constructs a local clearance-based visibility graph [26] using the robot's current pose, the local goal, and the abstracted LiDAR primitives. For circle primitives, 8 candidate waypoints are sampled evenly around the perimeter at a distance of the obstacle's radius plus a safety margin. For line primitives, candidate points are sampled outward perpendicularly from the segment endpoints.

The A* algorithm evaluates the line-of-sight between these nodes to find the shortest collision-free path. This specific hybrid combination – extracting a visibility graph from instantaneous obstacle geometry and routing A* over it to seed a continuous vehicle optimizer – has been demonstrated to be highly effective at bypassing localized minima [21, 46].

Empirical testing revealed that while evaluating the visibility graph at every control frame allows for faster topological replanning, it increased the maximum computation time per frame from 300 ms to over 500 ms, threatening real-time control stability. Therefore, to respect strict CPU bounds, the local A* is utilized reactively: it only runs upon initial planner startup or when the safety systems detect that the current TEB trajectory has collided with an obstacle.

Custom Formulations of TEB Residuals

While standard kinematic, dynamic, time, and circular obstacle costs, as defined in Section 3.5, were utilized in the system, specific custom formulations had to be derived for linear obstacles and start conditions to maintain solver stability in Pyceres.

Line Obstacle Cost Calculating the exact shortest distance between two line segments introduces non-differentiable edge cases. To solve this and maintain smooth Jacobians for the Ceres solver, a custom residual was derived. The minimum distance from the trajectory segment (endpoints A, B) to the obstacle segment (endpoints C, D) is approximated using a LogSumExp (Softmin) function [7] over the four endpoint-to-segment distances ($d_A \dots d_D$):

$$d_{min} = -\frac{1}{\alpha} \ln \left(\sum_{k=1}^{4} \exp(-\alpha \cdot d_k) \right), \tag{4.1}$$

where $\alpha > 0$ is a configurable smoothing parameter that dictates how strictly the function approximates the true minimum. Using this distance calculation, the residual

is formulated as:

$$R_{line} = \begin{cases} \sqrt{w} \cdot (r_{safe} - d_{min}) & \text{if } r_{safe} > d_{min}, \\ 0 & \text{otherwise.} \end{cases} \tag{4.2}$$

where r_{safe} is the safety margin around the robot and w is the residual weight (analogous to the circular obstacle formulation in Section 3.5).

Starting Dynamics Cost For the first segment of the trajectory (from the current position to the first planned pose), there is no "previous" pose to reference – and as such, there cannot be three poses to calculate acceleration. If unconstrained, the solver might generate a trajectory that begins at a high velocity while the physical robot is stationary, implicitly demanding an infinite, impossible instantaneous acceleration.

To prevent this, a custom dynamic cost is implemented specifically for the starting position. It uses the robot's current linear and angular velocities (v_{curr}, ω_{curr}), retrieved directly from the state estimation sensors, to calculate the acceleration and angular acceleration costs for the first trajectory segment.

Jacobians and Ceres Problem Setup

The standard ROS implementation of TEB relies on C++ and the `g2o` library [25]. However, because the TEB objective function is fundamentally a sum of squared weighted residuals, it can be easily adapted into a general-purpose non-linear least-squares framework. For this custom Python architecture, Google's Ceres Solver [1] was chosen via the `pyceres` library.

While `g2o` uses predefined graph edges, Ceres allows for the manual definition of residual blocks. Because `pyceres` lacks the C++ compile-time automatic differentiation feature of the native Ceres library, the partial derivatives for all implemented residuals were calculated analytically and provided directly to the solver.

To ensure the optimization can run in real-time, an obstacle filter is applied before constructing the Ceres problem. Rather than evaluating every obstacle against every pose, the system pre-calculates distances and only adds residual blocks to the solver for obstacles posing an immediate collision threat to specific segments. All obstacle parameters are pre-extracted into arrays upon initialization, allowing for vectorized operations instead of slow iterative loops.

The problem is solved using the Sparse Normal Cholesky linear solver via `pyceres`. Because Python's Global Interpreter Lock (GIL) [52] serializes the execution of bytecode, parallelizing the solver's execution from a Python process actually degrades performance due to thread management overhead. Therefore, the Ceres solver is strictly configured to execute in a single-threaded mode. To maintain a predictable control rate, the solver is capped at a strict maximum number of iterations. The Sparse Normal Cholesky solver was selected because the TEB optimization problem produces a highly sparse Jacobian matrix, which this decomposition solves efficiently, as also demonstrated in the original TEB formulation [42].

Latency Compensation

The implementation uses a warm start approach, as described in Section 3.5.6, to maintain temporal coherence. However, because non-linear optimization introduces computational latency, the UGV physically moves while the solver runs. If the trajectory state was preserved blindly, the initial poses would be outdated by the time the next frame begins.

To resolve this, a latency compensation mechanism is implemented. Before the next optimization step begins, the system queries the most recent odometry estimate (filtered through EKF). The displacement $(\Delta x, \Delta y, \Delta\theta)$ that accumulated during the optimization is used to transform the trajectory matrix. This ensures the warm-started trajectory matches the robot's current pose before being passed to the solver. After these adjustments, the first pose in the trajectory is translated into linear (v) and angular (ω) velocities and published to the UGV's motor controllers.

Architectural Limitations

The implemented local planner overcomes the single-topology limitation of standard TEB by utilizing the local A* fallback to jump obstacles. However, this hybrid architecture introduces its own limitations. First, the topological jump is purely reactive: because A* is not run continuously due to computational constraints, topological shifts only occur after the current trajectory is completely invalidated by a collision check.

Second, the local A* visibility graph evaluates only static instantaneous obstacle positions; it does not explicitly predict the future velocities of dynamic agents. Consequently, if the robot is forced to stop by the safety system due to a closing gap, the instantaneous geometric space may be entirely blocked. In such cases, the A* algorithm cannot find a valid path and returns no seed for the TEB optimizer, leading to a standstill deadlock even if the pedestrian is actively moving out of the way. These limitations are explicitly evaluated in Chapter 5.

4.4.5 Parameter Selection

Algorithmic parameters for the navigation stack are derived from the physical kinematics of the UGV and the computational bounds of the onboard hardware.

Spatial and Obstacle Constraints

- *Safety and Critical Radii.* The system uses a dual-boundary approach to account for the physical footprint of the UGV and control latency. The "safety radius" is set to 1.3 m, establishing a soft buffer slightly larger than the UGV's length. The optimizer penalizes trajectories entering this zone. Additionally, a "critical stop radius" is defined at 0.5 m. If any obstacle breaches this distance, an immediate emergency stop is triggered.

- *Cluster Break Distance.* This threshold is derived from the robot's kinematic clearance requirement. Since the UGV cannot physically traverse gaps narrower than its own width plus a safety margin, the break distance is set to 1.4 m. Sequential LiDAR points separated by less than this threshold are merged into a single solid obstacle boundary.

- *LiDAR Filtering.* To prevent self-occlusion artifacts, a minimum scan range of 0.33 m is enforced, explicitly masking rays reflected by the UGV's chassis. A 1D median filter (window size 3) is subsequently applied to reject isolated single-ray noise without overly smoothing out the geometry of genuine narrow obstacles.

- *Virtual Barriers.* Constraining the local planner to paved surfaces requires explicit spatial bounds. Given that pedestrian paths in the target environment maintain a minimum width of approximately 4 meters, a 2.0 m parallel lateral offset is applied relative to the global A* route. This generates virtual line obstacles that restrict the trajectory optimizer to the pavement.

Kinodynamic Limits

- *Velocity and Acceleration.* The maximum linear velocity (v_{max}) is restricted to 0.5 m/s, and angular velocity (ω_{max}) to 0.75 rad/s. While the Husky is capable of higher speeds, limiting velocity prevents severe skid-steering drift and increases safety in a crowded environment. Linear acceleration (a_{max}) is limited to 0.1 m/s^2. This low acceleration ensures smooth motion and provides a more predictable behavior for pedestrians.

Computational Constraints

- *Trajectory Lookahead.* The "max trajectory distance" is set to 7 m. This provides enough foresight for the TEB algorithm to navigate around large obstacles without overloading the solver with distant, irrelevant waypoints.

- *Trajectory Resolution.* The trajectory resolution limits are bounded between 0.5 m and 2.0 m. This means that each segment of the trajectory is at least 0.5 m and at most 2.0 m long. This ensures the matrix size remains small enough for real-time calculation while maintaining enough pose density to accurately describe curves.

- *Maximum Solver Iterations.* The Ceres optimizer is capped at 5 iterations per control frame. While the original TEB implementation [42] utilized four outer loops of five iterations each, computing all 20 iterations within a single cycle introduces significant latency, especially when using Python. Delaying the control response to calculate a mathematically perfect trajectory poses a safety risk in dynamic environments. Instead, the system executes only a single loop of 5 iterations and outputs this intermediate solution immediately to maintain a high control rate. Because the algorithm employs a warm start strategy, the subsequent control frame continues refining the trajectory exactly where the previous one left off. This approach effectively distributes the optimization workload across control frames, ensuring real-time responsiveness without sacrificing long-term path convergence.

4.4.6 Safety Mechanisms

The reliance on a strictly 2D perception pipeline introduces inherent vertical blind spots – most notably the lack of vertical vision due to the 2D LiDAR. To ensure the safe operation of a heavy UGV in a public environment, a hierarchy of safety nets was implemented. If any of the following conditions are met, the system overrides the local planner and triggers an immediate software-level emergency stop (E-Stop), halting all movement.

- **Terrain Anomalies.** To compensate for the 2D LiDAR's inability to detect negative obstacles (drop-offs) or low-lying obstacles (curbs, rocks), the internal IMU continuously monitors pitch and roll variances. Exceeding predefined angular thresholds indicates terrain anomalies, triggering an emergency stop.

- **Proximity Breach.** If the geometric abstraction pipeline detects any obstacle within a critical stop distance, the system assumes an imminent or ongoing collision and stops.

- **Stale Sensors.** The controller monitors the timestamps of incoming LiDAR and odometry streams. If the sensors lag or timeout, the system stops to prevent the robot from executing outdated trajectories while driving blind.
- **Trajectory Collision Prediction.** The TEB optimizer may occasionally fail to converge on a safe path within the time limit. Before sending commands to the motors, the controller evaluates a predefined number of initial segments ("trajectory collision lookahead") of the generated trajectory. If any of these segments intersect with an obstacle, the trajectory is deemed unsafe. The same check, with lookahead increased to be equal to path length, also acts as the trigger for topological replanning: if the TEB trajectory intersects an obstacle, the warm start is discarded, and the local A* visibility-graph search is re-run to generate a completely new trajectory seed for the next control frame.
- **Path Deviation.** If the UGV is forced to dodge multiple obstacles and its position deviates beyond the established virtual corridor boundaries (with an added safety padding) from the global A* route, it is considered lost.
- **Localization Jumps.** While GPS may produce high-frequency errors, IMU and wheel odometry streams should remain continuous. Any detected spatial discontinuity in the odometry estimate significantly larger than the vehicle's maximum velocity indicates a localization failure, pausing execution until the filter stabilizes.
- **Motion Stall.** If the controller is actively publishing velocity commands, but the sensors report little to no actual speed for a continuous period of time, the system deduces that the UGV is mechanically stuck and stops attempting to drive forward.
- **Network and Operator Override.** The UGV operates autonomously but must remain under the ultimate supervision of a human operator using the off-board Visualization node. The UI publishes a continuous heartbeat signal over the local network. If the controller does not receive this signal for 5 seconds, it assumes the network connection has dropped. The robot automatically halts to ensure the operator never loses the ability to trigger a remote software cancellation.

It is important to distinguish between recoverable stops and unrecoverable deadlocks. In many proximity breach scenarios, the stop is recoverable: once a pedestrian clears the path, the planner continues generating valid commands, and the robot resumes navigation. However, if the local planner remains trapped after a safety stop, manual operator intervention is required. Additionally, the Husky's 2D LiDAR has a 270-degree field of view, leaving a blind spot at the rear. The current system cannot guarantee rear obstacle detection if the local planner executes a reverse maneuver.

4.5 Ethical and Safety Considerations

Deploying a 50 kg autonomous vehicle in a public space involves inherent safety risks. The primary ethical consideration is to ensure that the UGV does not pose a physical threat to pedestrians, pets, or property. As discussed, the system's perception is intentionally constrained by its minimal sensor suite. To mitigate the risk, the system design prioritizes safety over navigational efficiency. This is reflected in the strict velocity caps, the critical stop radius, and the extensive software-level safety

mechanisms described in 4.4.6. Quantitative benchmarking involving human-like dynamic agents is only conducted in simulation to eliminate physical risk.

4.6 Experimental Methodology

To evaluate the performance, safety, and resource efficiency of the proposed lightweight navigation architecture, a series of controlled quantitative experiments was conducted. While the system is designed for and qualitatively tested on physical hardware, the primary comparative benchmarking was executed within a simulation environment. Simulation allows for exact reproducibility, precise dynamic obstacle scripting, and isolated performance profiling without the physical constraints of battery degradation or unmeasured environmental hazards.

4.6.1 Simulation Environment Setup

The experiments were conducted using Gazebo Sim (version 8.9.0) [24]. To evaluate computational efficiency limits on constrained hardware, the simulation and controller stack were executed on an Intel Core i5-9300H (2.40 GHz) with 32 GB of RAM, representing a modest consumer-grade CPU.

A simulated representation of Stryiskyi Park was utilized to accurately reflect the semantic and spatial challenges of the target environment. The environment's global topology was driven by OSM data, while the physical terrain was modeled using a 2D satellite projection overlaid on a topographic heightmap [2]. To ensure the simulation accurately reflected the physical Husky A200, the simulated 2D LiDAR was constrained to the hardware's 270-degree range, and the simulated wheel odometry and IMU data were processed through the exact same EKF node as the physical robot.

A comparative evaluation against the ROS 2 Nav2 stack [32] was initially considered. However, stable integration was not achieved within the project constraints. The proposed system utilizes sparse OSM-derived waypoints and lightweight geometric obstacle abstractions, whereas Nav2 is fundamentally designed around dense costmaps and behavior-tree-based navigation. Preliminary integration attempts using DWB and MPPI controllers in the custom pipeline resulted in unstable behavior, including localized spinning or total failure to avoid obstacles. Therefore, the final evaluation focuses exclusively on the standalone performance, safety behavior, and computational footprint of the proposed custom architecture. Formal Nav2 benchmarking under identical lightweight constraints is left for future work.

4.6.2 Obstacle Modeling and Test Scenario

Because the core objective of the system is local obstacle avoidance, artificial geometric obstacles were injected into the simulated park environment along the known OSM paths. Dynamic agents (pedestrians) were represented by 0.5 m-diameter cylinders, sized to approximate the physical footprint and LiDAR signature of a human and controlled by Gazebo actors.

The evaluation was consolidated into a single, dynamic stress test. The UGV was tasked with navigating a 50-meter segment of the global path that was densely populated with these simulated pedestrians. The actors were scripted to move in repeated linear trajectories strictly perpendicular to the UGV's direction of travel.

This specific scenario was engineered to target the known vulnerabilities of the system of sticking to one side of deformation around an obstacle [43]. Perpendicularly moving obstacles force the TEB local planner into a continuous state of trajectory

deformation and test the robustness of the emergency stop safety nets by simulating uncooperative pedestrians repeatedly cutting across the robot's path.

To guarantee reproducibility, the dynamic obstacle course was generated using a custom Python script. First, real-world geographic coordinates of Stryiskyi Park were converted from WGS84 latitude and longitude into a local East-North-Up (ENU) Cartesian coordinate frame using a WGS84 ellipsoid approximation. Pedestrians were then distributed along a 50-meter segment of the path.

To simulate the unpredictable nature of crowd navigation, randomized parameters were applied to each actor. The walking speed was sampled uniformly from 0.1 to 0.26 m/s, the lateral walking distance was sampled uniformly from 6.0 to 9.0 m, and the longitudinal spacing between consecutive pedestrians varied from 2.0 to 6.0 m. A fixed random seed (42) was utilized during generation to ensure that the exact identical layout and timing of pedestrian motion were loaded for every simulation run. The complete parameters of the test scenario are summarized in Table 4.1.

Table 4.1: Dynamic Obstacle Course Parameters

Parameter	**Value**
Course length	50 m
Pedestrian spacing	2.0 – 6.0 m
Pedestrian speed	0.1 – 0.26 m/s
Pedestrian walking distance	6.0 – 9.0 m

4.6.3 Evaluation Metrics

The experimental evaluation focused on the performance of the local trajectory optimization and obstacle avoidance systems. The global routing component (the A* graph search over the OSM network) was excluded from benchmarking. A* is a deterministic and optimal algorithm [18] – given an admissible heuristic, it is proven to find the optimal path if one exists. Therefore, evaluating its success provides no insight. Instead, the metrics were designed to evaluate how efficiently and safely the system executed that predefined global path in the presence of dynamic, unmapped constraints.

The system was evaluated over 30 independent simulation runs. A run was considered successful if the robot reached the goal without driving into obstacles. Recoverable safety stops – where the robot halted for a passing pedestrian and then resumed – were counted as successful runs.

The following quantitative metrics were recorded:

- **Success Rate (%):** The percentage of experimental runs where the UGV successfully navigated to the target destination (within 1 meter).
- **Path Efficiency (Ratio):** Measured as the shortest-path distance generated by the global A* planner divided by the actual distance traveled by the UGV. A score of 1.0 implies the UGV followed the center of the path perfectly.
- **Travel Time (s):** The average time required to complete successful runs, reflecting the delay introduced by dynamic avoidance and safety stops.

Because the evaluation relies on a discrete number of Bernoulli trials (success or failure), the true success rate is estimated using a 95% Wilson score interval, which remains accurate for highly skewed proportions and small sample sizes.

The work prioritizes computational accessibility, therefore system telemetry was logged to measure resource efficiency:

- **Control Frequency (Hz):** The actual rate at which the TEB optimizer generated and published velocity commands, measured directly inside the controller node.
- **CPU Utilization (%) and Memory Footprint (MB):** System-level process metrics captured at 1 Hz intervals using the Python `psutil` library [39].

Finally, a qualitative Failure Mode Analysis was conducted. Failed runs and recoverable safety interventions were categorized to explicitly identify the architectural limitations of the elastic-band planning approach and the 2D perception constraints.

Chapter 5

Experiments and Results

5.1 Experimental Objective

The goal of the experimental evaluation was to assess the proposed lightweight navigation pipeline under dynamic obstacle conditions. As detailed in Section 4.6, the primary benchmarking was conducted in simulation using a 50-meter stress test course populated with perpendicularly moving pedestrians. This setup was chosen to explicitly test the limits of the TEB local planner's elastic deformation capabilities and the reliability of the system's safety nets. The evaluation focused on three primary aspects: navigation success, computational efficiency, and hardware-software integration.

5.2 Navigation Results

A total of 30 independent simulated runs were executed on the generated dynamic course. The UGV successfully completed the course in 25 runs, yielding a measured success rate of 83.3%. Accounting for the sample size, the 95% Wilson score interval estimates the true system reliability to fall between 66% and 93%. Table 5.1 details the distribution of outcomes, including specific failure modes and recoverable safety interventions.

Table 5.1: Summary of Navigation Outcomes

Outcome	Count	Percentage
Successful runs	**25**	**83.3%**
With recoverable safety events	9	30.0%
Failed runs	**5**	**16.7%**
Squeeze/deadlock failures	4	13.3%
Rear blind-spot collision failure	1	3.3%

Out of the 25 successful runs, 9 runs included at least one recoverable safety event. In these instances, a moving pedestrian entered the critical stop radius of the robot, triggering the proximity breach safety net. The robot halted all motion ($v = 0, \omega = 0$) according to its safety constraints. While the UGV was entirely stationary, a simulated pedestrian's trajectory intersected the robot's physical footprint. Because the UGV was acting as a static environmental obstacle at the time of contact and did not induce the collision, these events were not categorized as failures. The safety hierarchy functioned exactly as intended, preventing robot-caused collisions, and the vehicle ultimately resumed its optimized trajectory once the pedestrian passed to complete its navigational objective.

5.2.1 Travel Time and Path Efficiency

Motion metrics were calculated exclusively across the 25 successful runs, as shown in Table 5.2. To provide a comprehensive representation of system performance without the distortion of extreme outliers, the distribution of results is reported using means, standard deviations, quartiles (Q1, Q3), and the 5th and 95th percentiles (P5, P95).

Table 5.2: Travel Time and Path Efficiency for Successful Runs

Metric	**Mean**	**Std.**	**P5**	**Q1**	**Median**	**Q3**	**P95**
Travel Time (s)	145.73	33.54	110.60	128.80	140.40	150.10	225.30
Path Efficiency	0.658	0.089	0.490	0.616	0.656	0.717	0.788

The mean travel time was 145.73 s, with a standard deviation of 33.54 s. This high variance strongly reflects the dynamic nature of the environment: runs where pedestrians crossed unfavorably required the UGV to execute extensive detours or pause for several seconds, whereas favorable timing allowed for near-continuous forward progress.

The mean path efficiency was 0.658. Because this metric is the ratio of the shortest A* path to the actual executed distance, the result indicates that the robot traveled approximately 52% further than the ideal route. This extra distance is entirely expected, as the local planner was forced into heavy lateral maneuvering to avoid the dense pedestrian traffic.

5.3 Computational Performance

A primary objective of this thesis was to demonstrate that autonomous navigation can be achieved with low resource overhead. Telemetry was logged continuously during the simulation to profile the Python-based controller execution. The results are summarized in Table 5.3.

Table 5.3: Computational Resource Usage and Control Frequency

Metric	**Mean**	**Std.**	**P5**	**Q1**	**Median**	**Q3**	**P95**
Control Freq. (Hz)	9.19	2.24	3.69	9.88	9.99	10.03	10.36
CPU Utilized (%)	9.26	2.43	5.00	7.50	9.49	11.63	11.99
Memory (MB)	292.99	3.31	287.34	291.52	293.92	295.71	297.29

The TEB controller was designed to target an update rate of 10 Hz. The measured median frequency was 9.99 Hz, with an interquartile range (Q1 to Q3) tightly bounded between 9.88 Hz and 10.03 Hz, confirming that the system reliably met the intended cycle time. The 5th percentile value (3.69 Hz) indicates occasional optimization latency, corresponding to mathematically complex solver frames where multiple dynamic obstacles entered the prediction horizon simultaneously. This metric represents the worst-case latency under heavy obstacle load, which is a critical safety-relevant constraint.

Resource consumption remained exceptionally light. Process CPU utilization averaged just 9.26%, with a 95th percentile of 11.99% and an absolute observed peak of 12.36% on the test system. It is worth noting that while the simulation and profiling were executed on a consumer-grade mobile CPU, the physical UGV is equipped with a comparable embedded processor; therefore, these metrics validate the computational feasibility of the pipeline on the target hardware. Memory footprint demonstrated

extreme stability, averaging 292.99 MB with a maximum standard deviation of only 3.31 MB, confirming that geometric obstacle abstraction effectively bypasses the heavy RAM requirements associated with dense costmaps or SLAM pipelines.

5.4 Failure Mode Analysis

The five failed runs provided critical insights into the inherent limitations of the proposed architecture. The failures were categorized into two distinct mechanisms: squeeze deadlocks and blind-spot collisions.

5.4.1 Squeeze and Deadlock Failures

Four of the five failures occurred when the UGV attempted to navigate between a pedestrian and the static virtual boundary representing the edge of the path. As the pedestrian moved laterally, the free space collapsed. The TEB optimizer continuously deformed the trajectory, squeezing the robot closer to the curb to maintain distance from the pedestrian. Eventually, the robot's physical proximity to the curb (and the pedestrian) breached the critical stop radius. The safety layer correctly intervened, halting the vehicle to prevent a collision.

However, once the pedestrian passed and the path forward was technically clear, the UGV remained permanently stalled. Because the robot had been pushed so close to the curb before stopping, its resting position was still within the critical safety margin of the static boundary. The safety net continued to classify the robot's state as an ongoing proximity breach with the curb.

This behavior confirms a structural limitation. A* replanning is only triggered when TEB path intersects an obstacle to ensure smooth motion. On the other hand, because TEB optimizes by iteratively deforming a continuous trajectory, it struggles to abandon its current topological class when getting trapped in a shrinking corridor. Rösmann et al. [41] addressed this exact deadlock by introducing an approach that simultaneously maintains and optimizes alternative trajectories. While their work demonstrated that a sampling-based exploration strategy can identify alternative routes with minimal CPU overhead when parallelized, integrating this multi-topology multi-threaded tracking into the custom PyCeres solver fell outside the scope of this thesis.

5.4.2 Rear Blind-Spot Collision

The final failure occurred during an attempted reverse maneuver. Faced with a similar squeeze scenario, the TEB optimizer generated a trajectory that commanded the robot to drive backward to escape the collapsing space. Unfortunately, a pedestrian happened to be located directly behind the vehicle. Because the Husky relies exclusively on a single 2D LiDAR with a 270-degree forward-facing field of view, the rear region was completely unobserved by the perception pipeline. The UGV therefore executed a kinodynamically valid, but physically disastrous, reversing trajectory into a blind spot, resulting in an unrecoverable collision.

5.5 Qualitative Hardware Validation

While the primary quantitative benchmarking was conducted in simulation to guarantee reproducibility, a preliminary physical deployment was executed using the real Clearpath Husky A200 platform. The objective of this test was to qualitatively validate

the hardware-software integration, specifically confirming that the perception pipeline processes real-world sensor noise and that the motor controllers correctly execute the generated TEB trajectories.

Because physical testing took place in an indoor laboratory setting, GPS and OSM global routing were unavailable. The test was therefore designed to evaluate the local trajectory optimization and sensor fusion in isolation. The EKF was configured to rely entirely on the IMU and wheel odometry for state estimation, while the local planner was provided with a relative 10-meter Cartesian goal directly ahead of the UGV.

Four physical square obstacles, each approximately 1.5 m in length, were placed in a staggered pattern between the robot and the goal. The maximum linear velocity was constrained to a conservative 0.2 m/s for safety during the initial indoor deployment.

The UGV successfully navigated the course. The custom perception pipeline accurately clustered the raw Hokuyo 2D LiDAR data and abstracted the physical boxes into line and circle primitives. The TEB optimizer continuously generated kinodynamically feasible maneuvers, successfully steering the robot around the staggered obstacles and reaching the target destination without triggering the emergency stop safety nets.

While a subsequent, unrelated technical issue with the Husky A200 platform prevented the execution of comprehensive outdoor quantitative testing, this physical run serves as a proof of concept. It must be acknowledged that navigating around four static obstacles at 0.2 m/s indoors is a significantly less demanding scenario than the dynamic outdoor simulation. Nonetheless, it confirms that the underlying ROS 2 architecture functions correctly on physical hardware, that the computational latency of the Python-based Ceres solver is acceptable for real-time control, and that the abstraction of real, noisy LiDAR data into geometric primitives is a viable approach for physical obstacle avoidance.

5.6 Conclusions

The experimental results validate the core hypothesis of the thesis: safe, autonomous point-to-point navigation in semi-structured environments is feasible utilizing only lightweight, geometric obstacle abstractions and minimal sensor inputs. The qualitative indoor hardware validation confirmed that the pipeline successfully translates real sensor data into physical motor commands. Furthermore, the quantitative simulation stress test demonstrated an 83.3% success rate in a deliberately adverse dynamic environment, achieved using under 13% peak CPU utilization and 300 MB of RAM. This underscores the efficiency of bypassing dense visual SLAM pipelines, which can consume up to $4 - 16$ GB of RAM [45]. The robustness of the hierarchical safety nets was particularly evident, recovering from 9 distinct dynamic contact events.

However, the stress test also explicitly highlighted the boundary limits of this minimal approach. The squeeze failures demonstrate that while the hybrid architecture successfully handles topological replanning, it is vulnerable to safety lockouts. Furthermore, the blind-spot failure clearly establishes that while 2D perception is computationally efficient, executing bidirectional trajectories mandates either 360-degree sensor coverage or strict forward-only motion constraints.

Chapter 6

Conclusions

The core objective of this research was to investigate whether a minimal, low-cost sensor suite – comprising a 2D LiDAR, GPS, IMU, and wheel encoders – combined with semantic environmental data and a CPU-optimized trajectory planner is sufficient for safe, autonomous UGV navigation in semi-structured outdoor environments. By balancing the need for capable outdoor autonomy with the limitations of constrained hardware, this thesis successfully demonstrated that heavy sensors and RAM-intensive SLAM algorithms are not strict prerequisites for reliable point-to-point navigation.

The global path planning objective was fulfilled by constructing a semantic OpenStreetMap environment graph and applying the deterministically optimal A* algorithm over it. By filtering OSM access tags and surface materials, collapsing multi-segment physical paths into navigable edges, and projecting geodetic coordinates into a local Cartesian frame, the system successfully generated metric-accurate global routes. This entirely eliminated the need to build or maintain a spatial map at runtime, validating the use of existing semantic data for lightweight global routing.

The obstacle detection objective was fulfilled through a custom 2D LiDAR perception pipeline. Raw polar scans were filtered and clustered based on physical clearance thresholds, abstracting the environment into compact Cartesian geometric primitives – lines and circles – using Split-and-Merge and an adapted circumcenter fitting technique. Empirically, this abstraction constrained the system's memory footprint to an average of 293 MB of RAM, standing in stark contrast to the $4 - 16$ GB typically demanded by dense visual SLAM pipelines [45].

The local navigation objective was accomplished by integrating a dual-EKF localization setup with a hybrid local planner, utilizing a dynamic visibility-graph A* search for initial path generation and a custom Python-based Timed Elastic Band optimizer for trajectory execution. The PyCeres solver was optimized for real-time execution through analytical Jacobians, temporal warm-starting, and explicit latency compensation, operating under the constraints of single-threaded execution. In a highly dynamic, pedestrian-dense simulation stress test, the local planner achieved an 83.3% success rate while maintaining a 9.99 Hz median control frequency and demanding less than 13% peak CPU utilization on a consumer-grade mobile processor, demonstrating that kinodynamically feasible MPC-style optimization is achievable within strict hardware bounds.

Taken together, these results demonstrate that robust obstacle avoidance and trajectory optimization can be achieved without a dedicated GPU or heavy 3D sensors, enabling the deployment of accessible, energy-efficient UGV platforms. Practically, this lightweight architecture is well suited for operations such as automated park maintenance or campus delivery in semi-structured pedestrian zones. The system's safety architecture – evidenced by the successfully recovered dynamic contact events during testing – further demonstrates that a hierarchical software-level emergency stop

approach provides a practically sound safety net even when the primary planner's prediction horizon is breached.

The failure mode analysis, however, outlines the current architectural limitations and informs directions for further research. The deadlock failures confirm that a single-topology elastic band planner struggles in shrinking corridors, and the logical next step is the integration of an extended TEB formulation that simultaneously maintains and optimizes multiple candidate trajectories [41], allowing the solver to select an alternative route around uncooperative agents rather than becoming trapped in a single corridor. Another possibility is to improve the performance of TEB and the dynamic visibility-graph A* to allow more frequent replanning. The rear-collision failure underscores a critical vulnerability of unidirectional 2D perception: safe bidirectional motion requires either 360-degree sensor coverage or enforcement of a strict forward-only kinematic constraint. Finally, while the 83.3% success rate is promising, the 95% Wilson score interval ($66\% - 93\%$) remains broad due to the sample size of 30 runs. Future work should encompass larger-scale quantitative trials and physical outdoor benchmarking to tighten these bounds and explicitly measure EKF dead-reckoning drift under severe GPS occlusion, such as that induced by dense tree canopies.

Bibliography

[1] Sameer Agarwal, Keir Mierle, and The Ceres Solver Contributors. *Ceres Solver*. 2012. URL: http://ceres-solver.org.

[2] Sai Aravind. *Gazebo Terrain Generator*. 2026. URL: https://github.com/saiaravind19/gazebo_terrain_generator.

[3] Kai O. Arras, Oscar Martinez Mozos, and Wolfram Burgard. "Using Boosted Features for the Detection of People in 2D Range Data". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2007, pp. 3402–3407. DOI: 10.1109/ROBOT.2007.363998.

[4] Geoff Boeing. "OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks". In: *Computers, Environment and Urban Systems* 65 (2017), pp. 126–139. DOI: 10.1016/j.compenvurbsys.2017.05.004.

[5] Johann Borenstein and Yoram Koren. "Obstacle avoidance with ultrasonic sensors". In: *IEEE Journal on Robotics and Automation* 4.2 (1988), pp. 213–218. DOI: 10.1109/56.2085.

[6] Johann Borenstein and Yoram Koren. "Real-time obstacle avoidance for fast mobile robots". In: *IEEE Transactions on Systems, Man, and Cybernetics* 19.5 (1989), pp. 1179–1187. DOI: 10.1109/21.44033.

[7] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge: Cambridge University Press, 2004. DOI: 10.1017/CBO9780470011515.

[8] Martin Brossard, Axel Barrau, and Silvère Bonnabel. "AI-IMU Dead-Reckoning". In: *IEEE Transactions on Intelligent Vehicles* 5.4 (2020), pp. 585–595. DOI: 10.1109/TIV.2020.2980758.

[9] Angelo Nikko Catapang and Manuel Ramos. "Obstacle Detection using a 2D LIDAR System for an Autonomous Vehicle". In: *Proceedings of the IEEE 6th International Conference on Control System, Computing and Engineering (ICCSCE)*. 2016, pp. 441–445. DOI: 10.1109/ICCSCE.2016.7893614.

[10] Marcello Cellina, Matteo Corno, and Sergio Matteo Savaresi. "LiDAR-Based Vehicle Detection and Tracking for Autonomous Racing". In: *arXiv preprint arXiv:2501.14502* (2025).

[11] Animesh Chakravarthy and Debasish Ghose. "Obstacle avoidance in a dynamic environment: a collision cone approach". In: *IEEE Transactions on Systems, Man, and Cybernetics, Part A: Systems and Humans* 28.5 (1998), pp. 562–574. DOI: 10.1109/3468.709600.

[12] Clearpath Robotics. *Husky A200 User Manual*. Accessed: 2026-04-23. URL: https://clearpathrobotics.com/husky-unmanned-ground-vehicle-robot.

[13] Yan Du et al. "Group Surfing: A Pedestrian-Based Approach to Sidewalk Robot Navigation". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2019, pp. 4418–4424. DOI: 10.1109/ICRA.2019.8793608.

[14] O. Dyhdalovych, A. Yaroshevych, O. Kapshii, et al. "Particle filter-based BLE and IMU fusion algorithm for indoor localization". In: *Telecommunication Systems* 88 (2025), p. 9. DOI: 10.1007/s11235-024-01230-6.

[15] Riccardo Enrico, Mauro Mancini, and Elisa Capello. "Comparison of NMPC and GPU-Parallelized MPPI for Real-Time UAV Control on Embedded Hardware". In: *Applied Sciences* 15.16 (2025). ISSN: 2076-3417. DOI: 10.3390/app15169114. URL: https://www.mdpi.com/2076-3417/15/16/9114.

[16] Mona Forsman et al. "Bias of cylinder diameter estimation from ground-based laser scanners with different beam widths: A simulation study". In: *ISPRS Journal of Photogrammetry and Remote Sensing* 135 (2018), pp. 84–92. DOI: 10.1016/j.isprsjprs.2017.11.013.

[17] Dieter Fox, Wolfram Burgard, and Sebastian Thrun. "The dynamic window approach to collision avoidance". In: *IEEE Robotics & Automation Magazine* 4.1 (1997), pp. 23–33. DOI: 10.1109/100.580977.

[18] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths". In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107. DOI: 10.1109/TSSC.1968.300136.

[19] Michael Hentschel and Bernardo Wagner. "Autonomous Robot Navigation Based on OpenStreetMap Geodata". In: *Proceedings of the IEEE Conference on Emerging Technologies and Factory Automation (ETFA)*. 2010, pp. 1–4. DOI: 10.1109/ETFA.2010.5625092.

[20] Hokuyo Automatic Co., Ltd. *UTM-30LX-EW Scanning Laser Range Finder Specification (Drawing No. C-42-3785)*. Accessed: 2026-04-27. Dec. 2012. URL: https://www.hokuyo-aut.jp.

[21] Karthik Karur et al. "A survey of path planning algorithms for mobile robots". In: *Vehicles* 3.3 (2021), pp. 448–468. DOI: 10.3390/vehicles3030027.

[22] Oussama Khatib. "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots". In: *The International Journal of Robotics Research* 5.1 (1986), pp. 90–98. DOI: 10.1177/027836498600500106.

[23] Jihoon Kim, Hyungjin Jeong, and Donghun Lee. "Single 2D lidar based follow-me of mobile robot on hilly terrains". In: *Journal of Mechanical Science and Technology* 34 (2020), pp. 3025–3034. DOI: 10.1007/s12206-020-0835-7.

[24] Nathan Koenig and Andrew Howard. "Design and use paradigms for Gazebo, an open-source multi-robot simulator". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Vol. 3. IEEE, 2004, pp. 2149–2154. DOI: 10.1109/IROS.2004.1389727.

[25] Rainer Kümmerle et al. "g2o: A General Framework for Graph Optimization". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2011, pp. 3607–3613. DOI: 10.1109/ICRA.2011.5979949.

[26] Steven M. LaValle. *Planning Algorithms*. Available at http://planning.cs.uiuc.edu/. Cambridge, U.K.: Cambridge University Press, 2006. URL: http://planning.cs.uiuc.edu/.

[27] Steven Macenski et al. "Robot Operating System 2: Design, architecture, and uses in the wild". In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074. URL: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074.

[28] Anthony Mandow et al. "Experimental Kinematics for Wheeled Skid-Steer Mobile Robots". In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2007, pp. 1222–1227. DOI: 10.1109/IROS.2007.4399139.

[29] Chen Min et al. "Autonomous Driving in Unstructured Off-Road Environments: How Far Have We Come?" In: *Journal of Field Robotics* (2024). DOI: 10.1002/rob.70154.

[30] Javier Minguez, Florent Lamiraux, and Jean-Paul Laumond. "Motion Planning and Obstacle Avoidance". In: *Springer Handbook of Robotics*. Springer, 2008, pp. 827–852. DOI: 10.1007/978-3-540-30301-5_36.

[31] Thomas Moore and Daniel Stouch. "A Generalized Extended Kalman Filter Implementation for the Robot Operating System". In: *Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS)*. Springer, 2015, pp. 335–348. DOI: 10.1007/978-3-319-08338-4_25.

[32] Nav2 Project. *Nav2 Controller Plugins*. 2024. URL: https://docs.nav2.org/configuration/index.html#controller-plugins (visited on 05/15/2024).

[33] Viet Nguyen et al. "A Comparison of Line Extraction Algorithms Using 2D Laser Rangefinder for Indoor Mobile Robotics". In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2005, pp. 1929–1934. DOI: 10.1109/IROS.2005.1545234.

[34] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. 2nd ed. Springer, 2006.

[35] Anish Pandey, Shweta Pandey, and Dayal R. Parhi. "Mobile robot navigation and obstacle avoidance techniques: A review". In: *International Journal of Control, Automation and Systems* 15.2 (2017), pp. 968–982. DOI: 10.1007/s12555-014-0456-z.

[36] Ruslan Partsey et al. "Is Mapping Necessary for Realistic PointGoal Navigation?" In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022, pp. 17232–17241.

[37] Phidgets Inc. *PhidgetSpatial 3/3/3 Basic – SKU 1042_0*. https://phidgets.com/?prodid=1025. Product page and datasheet. Accessed: 2024. 2024.

[38] Sean Quinlan and Oussama Khatib. "Elastic bands: Connecting path planning and control". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 1993, pp. 802–807. DOI: 10.1109/ROBOT.1993.291936.

[39] Giampaolo Rodola. *psutil: Cross-platform lib for process and system monitoring in Python*. https://github.com/giampaolo/psutil. 2024.

[40] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2020.

[41] Christoph Rösmann, Frank Hoffmann, and Torsten Bertram. "Integrated online trajectory planning and optimization in distinctive topologies". In: *Robotics and Autonomous Systems* 88 (2017), pp. 142–153. ISSN: 0921-8890. DOI: https://doi.org/10.1016/j.robot.2016.11.007. URL: https://www.sciencedirect.com/science/article/pii/S0921889016300495.

[42] Christoph Rösmann et al. "Efficient Trajectory Optimization Using a Sparse Model". In: *Proceedings of the European Conference on Mobile Robots (ECMR)*. 2013, pp. 138–143.

[43] Christoph Rösmann et al. "Trajectory Modification Considering Dynamic Constraints of Autonomous Robots". In: *Proceedings of the 7th German Conference on Robotics (ROBOTIK)*. 2012, pp. 1–6.

[44] Paul-Edouard Sarlin, Philipp Lindenberger, and Shaohui Liu. *pyceres: Minimal Python bindings for the Ceres Solver*. https://github.com/cvg/pyceres. 2025.

[45] Fabian Schmidt et al. "Visual-Inertial SLAM for Unstructured Outdoor Environments: Benchmarking the Benefits and Computational Costs of Loop Closing". In: *Journal of Field Robotics* (2025). DOI: 10.1002/rob.22581.

[46] Saeid Sedighi, Duong-Van Nguyen, and Klaus-Dieter Kuhnert. "Guided Hybrid A-star Path Planning Algorithm for Valet Parking Applications". In: *2019 5th International Conference on Control, Automation and Robotics (ICCAR)*. 2019, pp. 570–575. DOI: 10.1109/ICCAR.2019.8813752.

[47] Volkan Sezer and Metin Gokasan. "A novel obstacle avoidance algorithm: "Follow the Gap Method "". In: *Robotics and Autonomous Systems* 60.9 (2012), pp. 1123–1134. DOI: 10.1016/j.robot.2012.05.021.

[48] Pascal Stahr, Jochen Maaß, and Henner Gärtner. "Application of Path Planning for a Mobile Robot Assistance System Based on OpenStreetMap Data". In: *Robotics* 12.4 (2023), p. 113. DOI: 10.3390/robotics12040113.

[49] Massimo Stefanoni et al. "A Survey on the Main Techniques Adopted in Indoor and Outdoor Localization". In: *Electronics* 14.10 (2025), p. 2069. DOI: 10.3390/electronics14102069.

[50] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, 2005.

[51] u-blox AG. *NEO-6 Data Sheet: u-blox 6 GPS Modules*. https://content.u-blox.com/sites/default/files/products/documents/NEO-6_DataSheet_(GPS.G6-HW-09005).pdf. Accessed: 2024. 2011.

[52] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009. ISBN: 1441412697.

[53] Grady Williams et al. "Aggressive Driving with Model Predictive Path Integral Control". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2016, pp. 1433–1440. DOI: 10.1109/ICRA.2016.7487277.

[54] Oliver J. Woodman. *An Introduction to Inertial Navigation*. Tech. rep. UCAM-CL-TR-696. University of Cambridge, Computer Laboratory, 2007.

[55] João Xavier et al. "Fast Line, Arc/Circle and Leg Detection from Laser Scan Data in a Player Driver". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2005, pp. 3930–3935. DOI: 10.1109/ROBOT.2005.1570721.